

Machine learning project on Automobiles Insurance Claim Fraud Detection. Presented by Group 28 Members are Asra Anees & Ali Abdullah

```
!pip install scikit-plot
```

```
Looking in indexes: https://pypi.org/simple, https://us-python.pkg.dev/colab-wheels/
Requirement already satisfied: scikit-plot in /usr/local/lib/python3.9/dist-packages
Requirement already satisfied: joblib>=0.10 in /usr/local/lib/python3.9/dist-packages
Requirement already satisfied: scikit-learn>=0.18 in /usr/local/lib/python3.9/dist-packages
Requirement already satisfied: matplotlib>=1.4.0 in /usr/local/lib/python3.9/dist-packages
Requirement already satisfied: scipy>=0.9 in /usr/local/lib/python3.9/dist-packages
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.9/dist-packages
Requirement already satisfied: pillow>=6.2.0 in /usr/local/lib/python3.9/dist-packages
Requirement already satisfied: importlib-resources>=3.2.0 in /usr/local/lib/python3.9/dist-packages
Requirement already satisfied: kiwisolver>=1.0.1 in /usr/local/lib/python3.9/dist-packages
Requirement already satisfied: numpy>=1.20 in /usr/local/lib/python3.9/dist-packages
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.9/dist-packages
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.9/dist-packages
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.9/dist-packages
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.9/dist-packages
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.9/dist-packages
Requirement already satisfied: threadpoolctl>=2.0.0 in /usr/local/lib/python3.9/dist-packages
Requirement already satisfied: zipp>=3.1.0 in /usr/local/lib/python3.9/dist-packages
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.9/dist-packages (fr
```

Double-click (or enter) to edit

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from scipy.stats import zscore
from sklearn.preprocessing import PowerTransformer
from sklearn.preprocessing import LabelEncoder
from sklearn.preprocessing import StandardScaler
from statsmodels.stats.outliers_influence import variance_inflation_factor
from sklearn import datasets
from sklearn import metrics
from sklearn import model_selection
from scikitplot.metrics import plot_roc_curve
import warnings
warnings.filterwarnings('ignore')

# Loads data from a CSV file and returns a Pandas DataFrame"
def load_data(file_name):
    return pd.read_csv(file_name)

file_name = "/content/Auto_Insurance_Claims_Data.csv"
data_frame = load_data(file_name)
```

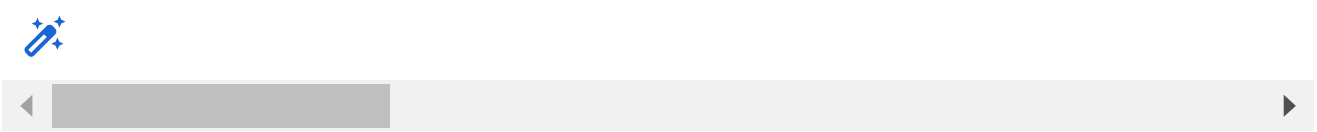
```
# summery of dataset
print(data_frame.info(verbose=False, null_counts=True))
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Columns: 40 entries, months_as_customer to _c39
dtypes: float64(2), int64(17), object(21)
memory usage: 312.6+ KB
None
```

```
data_frame.head()
```

	months_as_customer	age	policy_number	policy_bind_date	policy_state	policy_csl
0	328	48	521585	17-10-2014	OH	250/500
1	228	42	342868	27-06-2006	IN	250/500
2	134	29	687698	06-09-2000	OH	100/300
3	256	41	227811	25-05-1990	IL	250/500
4	228	44	367455	06-06-2014	IL	500/1000

5 rows × 40 columns



```
data_frame.tail()
```

	months_as_customer	age	policy_number	policy_bind_date	policy_state	policy_csl
995	3	38	941851	16-07-1991	OH	500/1000
996	285	41	186934	05-01-2014	IL	100/500
997	130	34	918516	17-02-2003	OH	250/500
998	458	62	533940	18-11-2011	IL	500/1000
999	456	60	556080	11-11-1996	OH	250/500

5 rows × 40 columns



```
data_frame.columns
```

```
Index(['months_as_customer', 'age', 'policy_number', 'policy_bind_date',
      'policy_state', 'policy_csl', 'policy_deductable',
      'policy_annual_premium', 'umbrella_limit', 'insured_zip', 'insured_sex',
      'insured_education_level', 'insured_occupation', 'insured_hobbies',
      'insured_relationship', 'capital-gains', 'capital-loss',
```

```
'incident_date', 'incident_type', 'collision_type', 'incident_severity',
'authorities_contacted', 'incident_state', 'incident_city',
'incident_location', 'incident_hour_of_the_day',
'number_of_vehicles_involved', 'property_damage', 'bodily_injuries',
'witnesses', 'police_report_available', 'total_claim_amount',
'injury_claim', 'property_claim', 'vehicle_claim', 'auto_make',
'auto_model', 'auto_year', 'fraud_reported', '_c39'],
dtype='object')
```

```
# dataset overview
```

```
data_frame.info()
```

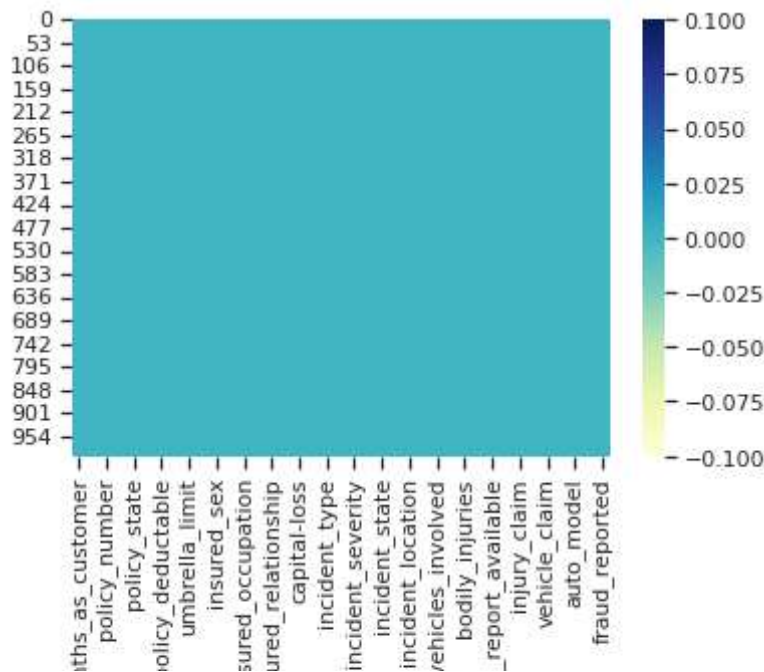
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 40 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   months_as_customer                    1000 non-null   int64
1   age                                   1000 non-null   int64
2   policy_number                         1000 non-null   int64
3   policy_bind_date                      1000 non-null   object
4   policy_state                          1000 non-null   object
5   policy_csl                            1000 non-null   object
6   policy_deductable                     1000 non-null   int64
7   policy_annual_premium                 1000 non-null   float64
8   umbrella_limit                        1000 non-null   int64
9   insured_zip                           1000 non-null   int64
10  insured_sex                           1000 non-null   object
11  insured_education_level               1000 non-null   object
12  insured_occupation                    1000 non-null   object
13  insured_hobbies                       1000 non-null   object
14  insured_relationship                  1000 non-null   object
15  capital-gains                         1000 non-null   int64
16  capital-loss                          1000 non-null   int64
17  incident_date                         1000 non-null   object
18  incident_type                         1000 non-null   object
19  collision_type                        1000 non-null   object
20  incident_severity                     1000 non-null   object
21  authorities_contacted                 1000 non-null   object
22  incident_state                        1000 non-null   object
23  incident_city                         1000 non-null   object
24  incident_location                     1000 non-null   object
25  incident_hour_of_the_day              1000 non-null   int64
26  number_of_vehicles_involved           1000 non-null   int64
27  property_damage                       1000 non-null   object
28  bodily_injuries                       1000 non-null   int64
29  witnesses                             1000 non-null   int64
30  police_report_available               1000 non-null   object
31  total_claim_amount                   1000 non-null   int64
32  injury_claim                          1000 non-null   int64
33  property_claim                        1000 non-null   int64
34  vehicle_claim                         1000 non-null   int64
35  auto_make                             1000 non-null   object
36  auto_model                           1000 non-null   object
37  auto_year                             1000 non-null   int64
38  fraud_reported                       1000 non-null   object
39  _c39                                 0 non-null      float64
dtypes: float64(2), int64(17), object(21)
memory usage: 312.6+ KB
```

```
# Dropping _c39 column because it have nul values
data_frame.drop("_c39",axis=1,inplace=True)
```

```
# check if there are any null values in the dataset.
data_frame.apply(lambda x: x.isnull().sum())
```

```
months_as_customer      0
age                      0
policy_number            0
policy_bind_date         0
policy_state             0
policy_csl               0
policy_deductable        0
policy_annual_premium    0
umbrella_limit           0
insured_zip              0
insured_sex              0
insured_education_level  0
insured_occupation       0
insured_hobbies           0
insured_relationship      0
capital-gains            0
capital-loss             0
incident_date            0
incident_type            0
collision_type           0
incident_severity        0
authorities_contacted    0
incident_state           0
incident_city            0
incident_location        0
incident_hour_of_the_day  0
number_of_vehicles_involved 0
property_damage          0
bodily_injuries          0
witnesses               0
police_report_available  0
total_claim_amount       0
injury_claim             0
property_claim           0
vehicle_claim            0
auto_make                0
auto_model               0
auto_year                0
fraud_reported           0
dtype: int64
```

```
# visuale representation of the null values
sns.heatmap(data_frame.isnull(), cmap="YlGnBu")
plt.show()
```



no null value in dataset

```

# examin datatypes of each coloumn
data_frame.dtypes

```

```

months_as_customer      int64
age                     int64
policy_number           int64
policy_bind_date        object
policy_state            object
policy_csl              object
policy_deductable       int64
policy_annual_premium   float64
umbrella_limit          int64
insured_zip             int64
insured_sex            object
insured_education_level object
insured_occupation      object
insured_hobbies         object
insured_relationship    object
capital-gains           int64
capital-loss            int64
incident_date           object
incident_type           object
collision_type          object
incident_severity       object
authorities_contacted   object
incident_state          object
incident_city           object
incident_location       object
incident_hour_of_the_day int64
number_of_vehicles_involved int64
property_damage         object
bodily_injuries         int64
witnesses              int64
police_report_available object
total_claim_amount      int64
injury_claim            int64

```

property_claim	int64
vehicle_claim	int64
auto_make	object
auto_model	object
auto_year	int64
fraud_reported	object
dtype:	object

```
#Listing the unique values from each column  
data_frame.nunique().to_frame("Unique Val")
```

policy_annual_premium	991
umbrella_limit	11
insured_zip	995
insured_sex	2
insured_education_level	7
insured_occupation	14
insured_hobbies	20
insured_relationship	6
capital-gains	338
capital-loss	354
incident_date	60
incident_type	4
collision_type	4
incident_severity	4

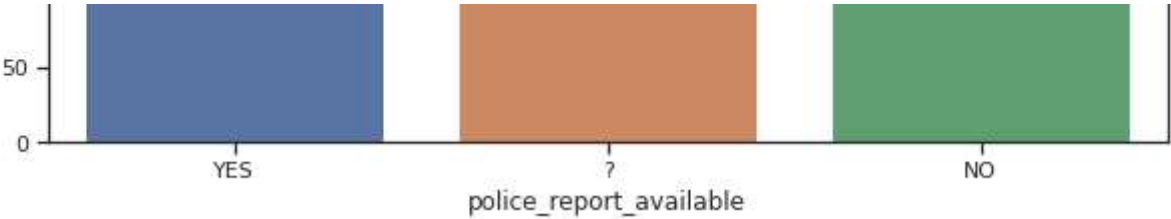
```
# drop the coloumn which have 1000 unique values
a= ["policy_number", "incident_location"]
data_frame.drop(a, axis=1,inplace=True)
```

```
data_frame.shape
```

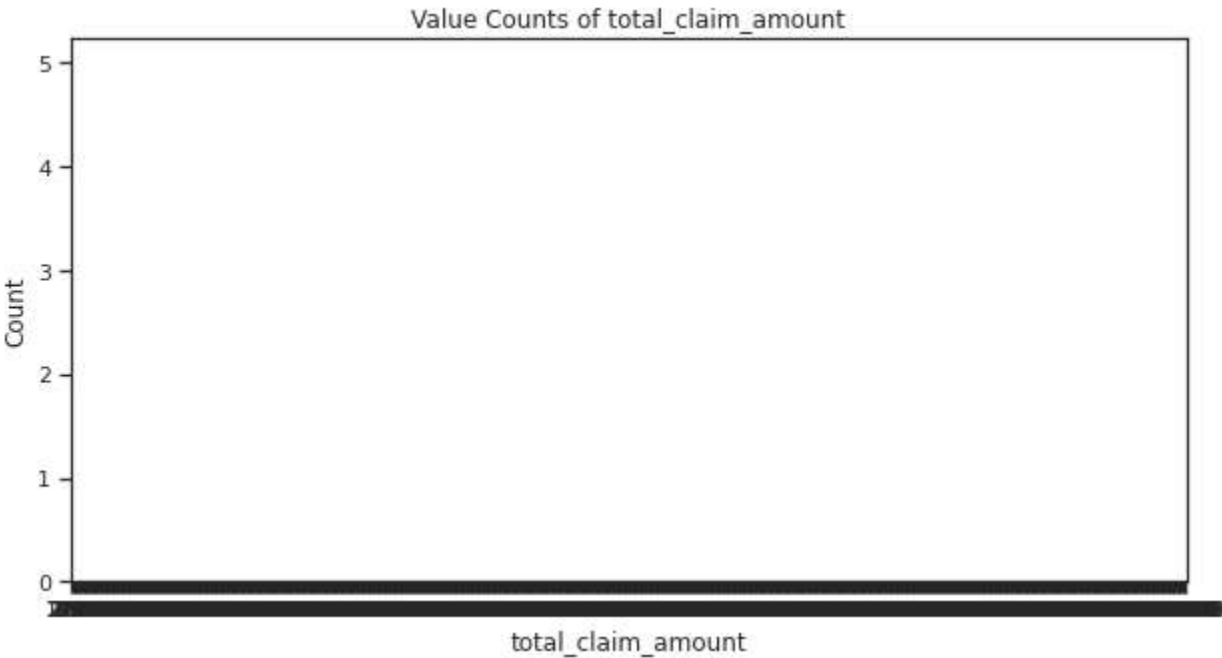
```
(1000, 37)
```

```
# checking the value counts for each column to detect any corrupted or unexpected data ent
for col in data_frame.columns:
    print(f"Column Name: {col}")
    print(data_frame[col].value_counts())
    print('*'*60)
```

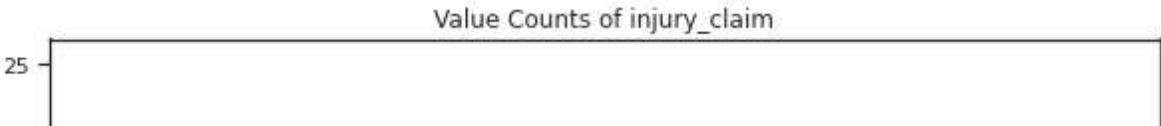
```
# Plot the value counts as a bar chart
plt.figure(figsize=(10,5))
sns.countplot(data=data_frame, x=col)
plt.title(f"Value Counts of {col}")
plt.xlabel(col)
plt.ylabel("Count")
plt.show()
```



```
Column Name: total_claim_amount
59400      5
2640       4
70400      4
4320       4
44200      4
..
65250      1
87100      1
6240       1
66600      1
67500      1
Name: total_claim_amount, Length: 763, dtype: int64
*****
```



```
Column Name: injury_claim
0          25
640         7
480         7
660         5
580         5
..
14840       1
6580        1
11820       1
16650       1
7500        1
Name: injury_claim, Length: 638, dtype: int64
*****
```



```
# Drop umbrella_limit col and insured_zip col as it is not necessary
a = ["insured_zip", "umbrella_limit"]
data_frame.drop(a, axis=1, inplace=True)
```

```
data_frame.shape
```

```
(1000, 35)
```

```
#Converting Date columns into datetime datatype so that Python can properly interpret the
data_frame[['policy_bind_date', 'incident_date']] = data_frame[['policy_bind_date', 'incid
```

```
data_frame.dtypes
```

```
months_as_customer      int64
age                      int64
policy_bind_date         datetime64[ns]
policy_state             object
policy_csl               object
policy_deductable        int64
policy_annual_premium    float64
insured_sex              object
insured_education_level  object
insured_occupation       object
insured_hobbies           object
insured_relationship     object
capital-gains            int64
capital-loss             int64
incident_date            datetime64[ns]
incident_type            object
collision_type           object
incident_severity        object
authorities_contacted    object
incident_state           object
incident_city            object
incident_hour_of_the_day int64
number_of_vehicles_involved int64
property_damage          object
bodily_injuries          int64
witnesses                int64
police_report_available  object
total_claim_amount       int64
injury_claim             int64
property_claim           int64
vehicle_claim            int64
auto_make                object
auto_model               object
auto_year                int64
fraud_reported           object
dtype: object
```

```
#Extracting the day, month, and year values from the date columns
```

```
date_cols = ['policy_bind_date', 'incident_date']
```

```
date_col_names = ['policy_bind', 'incident']
```

```
for col, name in zip(date_cols, date_col_names):
    data_frame[f"{name}_Day"] = pd.to_datetime(data_frame[col]).dt.day
    data_frame[f"{name}_Month"] = pd.to_datetime(data_frame[col]).dt.month
    data_frame[f"{name}_Year"] = pd.to_datetime(data_frame[col]).dt.year
```

```
# Dropping policy_bind_date and incident_date columns
data_frame.drop(["policy_bind_date", "incident_date"], axis=1, inplace=True)
```

```
data_frame.shape
```

```
(1000, 39)
```

```
data_frame.head()
```

	months_as_customer	age	policy_state	policy_csl	policy_deductable	policy_annua
0	328	48	OH	250/500	1000	
1	228	42	IN	250/500	2000	
2	134	29	OH	100/300	2000	
3	256	41	IL	250/500	2000	
4	228	44	IL	500/1000	1000	

```
5 rows × 39 columns
```



```
data_frame.shape
```

```
(1000, 39)
```

```
# Checking for missing values represented as "?" the we observe from value count
data_frame[data_frame.columns[(data_frame == '?').any()]].unique()
```

```
collision_type          4
property_damage         3
police_report_available 3
dtype: int64
```

```
# Calculating mode values of the columns
```

```
for col in ["collision_type", "property_damage", "police_report_available"]:
    mode_val = data_frame[col].mode()[0]
    print(f"The mode of {col} is: {mode_val}")
```

```
The mode of collision_type is: Rear Collision
The mode of property_damage is: ?
The mode of police_report_available is: ?
```

```
# Calculating value counts of the columns
for col in ["property_damage", "police_report_available"]:
    print(f"The value count of {col}:\n{data_frame[col].value_counts()}\n")

    The value count of property_damage:
    ?      360
    NO     338
    YES    302
    Name: property_damage, dtype: int64

    The value count of police_report_available:
    ?      343
    NO     343
    YES    314
    Name: police_report_available, dtype: int64

# Imputing missing values with mode values
a = data_frame['collision_type'].mode()[0]
data_frame['collision_type'] = data_frame['collision_type'].replace('?', a)
data_frame['property_damage'] = data_frame['property_damage'].replace('?', 'NO')
data_frame['police_report_available'] = data_frame['police_report_available'].replace('?',

# Verifying the value counts of the columns
for col in ["collision_type", "property_damage", "police_report_available"]:
    print(f"The value count of {col}:\n{data_frame[col].value_counts()}\n")

    The value count of collision_type:
    Rear Collision      470
    Side Collision      276
    Front Collision     254
    Name: collision_type, dtype: int64

    The value count of property_damage:
    NO      698
    YES     302
    Name: property_damage, dtype: int64

    The value count of police_report_available:
    NO      686
    YES     314
    Name: police_report_available, dtype: int64

# We noticed that the 'policy_csl' column has numerical data but its data type is 'object'
# To handle this, we will extract 'csl_per_person' and 'csl_per_accident' columns from 'po
data_frame[['person_csl', 'accident_csl']] = data_frame.policy_csl.str.split('/', expand=T
data_frame[['person_csl', 'accident_csl']] = data_frame[['person_csl', 'accident_csl']].ap

# We no longer need the policy_csl column after extracting csl_per_person and csl_per_acci
data_frame.drop("policy_csl", axis=1, inplace=True)
```

```
data_frame.dtypes
```

```
months_as_customer    int64
age                   int64
policy_state           object
policy_deductable      int64
policy_annual_premium  float64
insured_sex            object
insured_education_level object
insured_occupation     object
insured_hobbies         object
insured_relationship   object
capital-gains          int64
capital-loss           int64
incident_type          object
collision_type         object
incident_severity      object
authorities_contacted  object
incident_state         object
incident_city          object
incident_hour_of_the_day int64
number_of_vehicles_involved int64
property_damage        object
bodily_injuries         int64
witnesses              int64
police_report_available object
total_claim_amount     int64
injury_claim           int64
property_claim         int64
vehicle_claim          int64
auto_make              object
auto_model             object
auto_year              int64
fraud_reported         object
policy_bind_Day        int64
policy_bind_Month      int64
policy_bind_Year       int64
incident_Day           int64
incident_Month         int64
incident_Year          int64
person_csl             int64
accident_csl           int64
dtype: object
```

```
# calculate the vehicle age by subtracting the value of 'auto_year' column from 20
data_frame = data_frame.assign(Vehicle_Age=2022-data_frame["auto_year"]).pipe(lamb
```

```
# re-examine the unique values
data_frame.nunique().to_frame("Unique Val")
```

	Unique Val	
months_as_customer	391	
age	46	
policy_state	3	
policy_deductable	3	
policy_annual_premium	991	
insured_sex	2	
insured_education_level	7	
insured_occupation	14	
insured_hobbies	20	
insured_relationship	6	
capital-gains	338	
capital-loss	354	
incident_type	4	
collision_type	3	
incident_severity	4	
authorities_contacted	5	
incident_state	7	
incident_city	7	
incident_hour_of_the_day	24	
number_of_vehicles_involved	4	
property_damage	2	
bodily_injuries	3	
witnesses	4	
police_report_available	2	
total_claim_amount	763	
injury_claim	638	
property_claim	626	
vehicle_claim	726	
auto_make	14	
auto_model	39	
fraud_reported	2	
policy_bind_Day	31	

```

policy_bind_Month      12
# Drop incident_Year column because it has one unique value
data_frame.drop("incident_Year", axis=1, inplace=True)

incident_Day           22

# before processing overview of dataset
data_frame.shape

(1000, 39)

data_frame.head()

```

	months_as_customer	age	policy_state	policy_deductable	policy_annual_premium
0	328	48	OH	1000	1406.91
1	228	42	IN	2000	1197.22
2	134	29	OH	2000	1413.14
3	256	41	IL	2000	1415.74
4	228	44	IL	1000	1583.91

5 rows × 39 columns



```

# separate the numerical and categorical columns
column_dict = {col: data_frame[col].dtype for col in data_frame.columns}
cc = [col for col, dtype in column_dict.items() if dtype == 'object']
nc = [col for col, dtype in column_dict.items() if dtype != 'object']
print("\nCategorical columns are:\n", cc)
print("\Numerical columns are:\n", nc)

```

Categorical columns are:

```
['policy_state', 'insured_sex', 'insured_education_level', 'insured_occupation', 'ir
```

Numerical columns are:

```
['months_as_customer', 'age', 'policy_deductable', 'policy_annual_premium', 'capital
```

```

#Check the list of counts and unique values of target
ut= data_frame['fraud_reported'].unique()
tc = data_frame['fraud_reported'].value_counts()
print("\nUnique values in 'fraud_reported' column:\n", ut)
print("\nCount of each unique value in 'fraud_reported' column:\n", tc)

```

Unique values in 'fraud_reported' column:

```
['Y' 'N']
```

Count of each unique value in 'fraud_reported' column:

N 753

Y 247

Name: fraud_reported, dtype: int64

numerical coloumn statistical summary

s = data_frame.describe()

print("Statistical summary of columns:\n", s)

std	115.113174	9.140287	611.864673
min	0.000000	19.000000	500.000000
25%	115.750000	32.000000	500.000000
50%	199.500000	38.000000	1000.000000
75%	276.250000	44.000000	2000.000000
max	479.000000	64.000000	2000.000000

	policy_annual_premium	capital-gains	capital-loss \
count	1000.000000	1000.000000	1000.000000
mean	1256.406150	25126.100000	-26793.700000
std	244.167395	27872.187708	28104.096686
min	433.330000	0.000000	-111100.000000
25%	1089.607500	0.000000	-51500.000000
50%	1257.200000	0.000000	-23250.000000
75%	1415.695000	51025.000000	0.000000
max	2047.590000	100500.000000	0.000000

	incident_hour_of_the_day	number_of_vehicles_involved	bodily_injuries \
count	1000.000000	1000.000000	1000.000000
mean	11.644000	1.83900	0.992000
std	6.951373	1.01888	0.820127
min	0.000000	1.00000	0.000000
25%	6.000000	1.00000	0.000000
50%	12.000000	1.00000	1.000000
75%	17.000000	3.00000	2.000000
max	23.000000	4.00000	2.000000

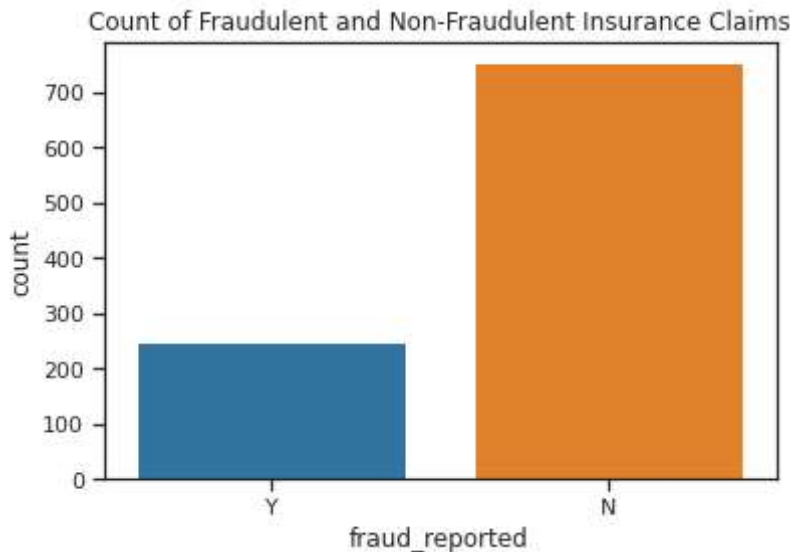
	witnesses ...	property_claim	vehicle_claim	policy_bind_Day \
count	1000.000000 ...	1000.000000	1000.000000	1000.000000
mean	1.487000 ...	7399.570000	37928.950000	15.448000
std	1.111335 ...	4824.726179	18886.252893	8.850176
min	0.000000 ...	0.000000	70.000000	1.000000
25%	1.000000 ...	4445.000000	30292.500000	8.000000
50%	1.000000 ...	6750.000000	42100.000000	16.000000
75%	2.000000 ...	10885.000000	50822.500000	23.000000
max	3.000000 ...	23670.000000	79560.000000	31.000000

	policy_bind_Month	policy_bind_Year	incident_Day	incident_Month \
count	1000.000000	1000.000000	1000.000000	1000.000000
mean	6.559000	2001.604000	13.08400	3.407000
std	3.391758	7.360391	10.44318	3.276291
min	1.000000	1990.000000	1.00000	1.000000
25%	4.000000	1995.000000	2.00000	1.000000
50%	7.000000	2002.000000	15.00000	2.000000
75%	9.000000	2008.000000	22.00000	5.000000
max	12.000000	2015.000000	31.00000	12.000000

std	161.603196	287.420547	6.015861
min	100.000000	300.000000	7.000000
25%	100.000000	300.000000	12.000000
50%	250.000000	500.000000	17.000000
75%	500.000000	1000.000000	22.000000
max	500.000000	1000.000000	27.000000

[8 rows x 22 columns]

```
#Plotting a bar graph to show the count of fraudulent insurance claims
sns.countplot(x="fraud_reported", data=data_frame, palette="tab10")
plt.title("Count of Fraudulent and Non-Fraudulent Insurance Claims")
plt.show()
```

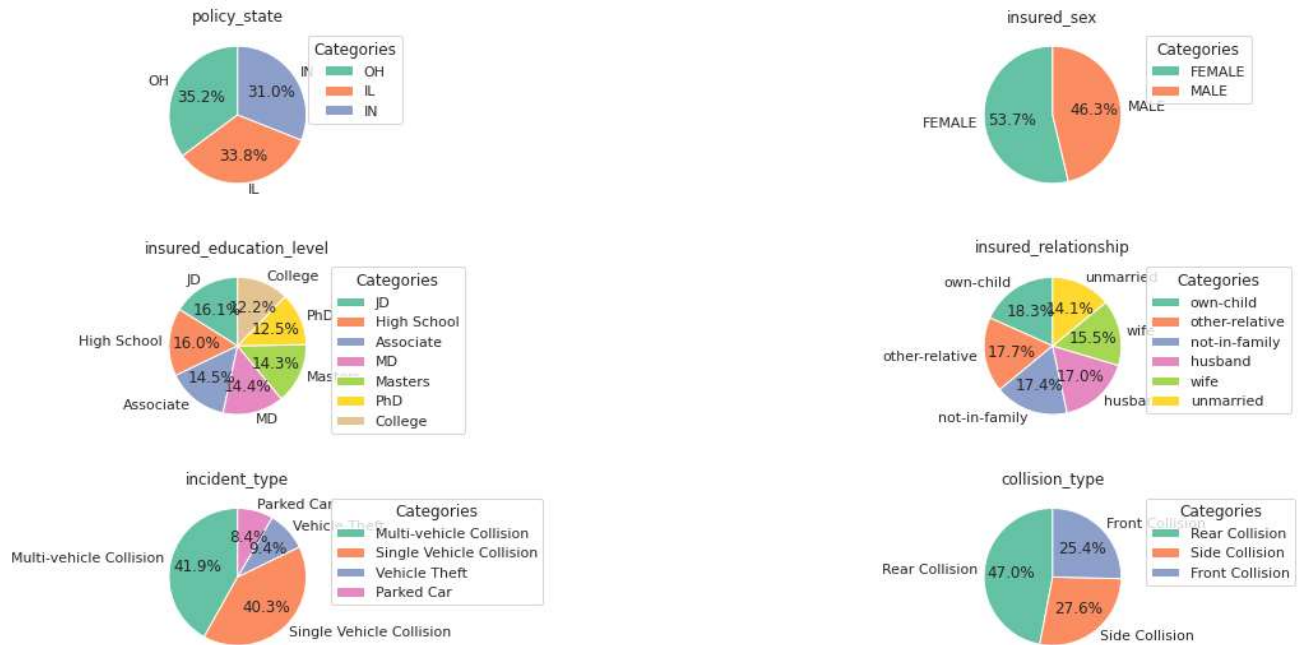


The targeted col are imbalance o balance it, we perform oversampling later on.

```
# generate pie charts
cols1 = ['policy_state', 'insured_sex', 'insured_education_level', 'insured_relatio

fig, axes = plt.subplots(nrows=4, ncols=2, figsize=(20, 10))
colors = sns.color_palette('Set2')
for ax, col in zip(axes.flatten(), cols1):
    data_frame[col].value_counts().plot(kind='pie', ax=ax, autopct='%1.1f%%', star
    ax.set(title=col, ylabel='')
    ax.legend(title='Categories', loc='best', bbox_to_anchor=(1, 0, 0.5, 1))

plt.tight_layout()
plt.show()
```



```
cols2 = ['insured_occupation', 'insured_hobbies', 'incident_severity', 'authoritie
```

```
fig, axes = plt.subplots(nrows=4, ncols=2, figsize=(20, 25))
```

```
colors = ['red', 'green', 'skyblue', 'orange', 'purple', 'olive', 'indigo', 'gray'
```

```
for ax, col, color in zip(axes.flatten(), cols2, colors):
```

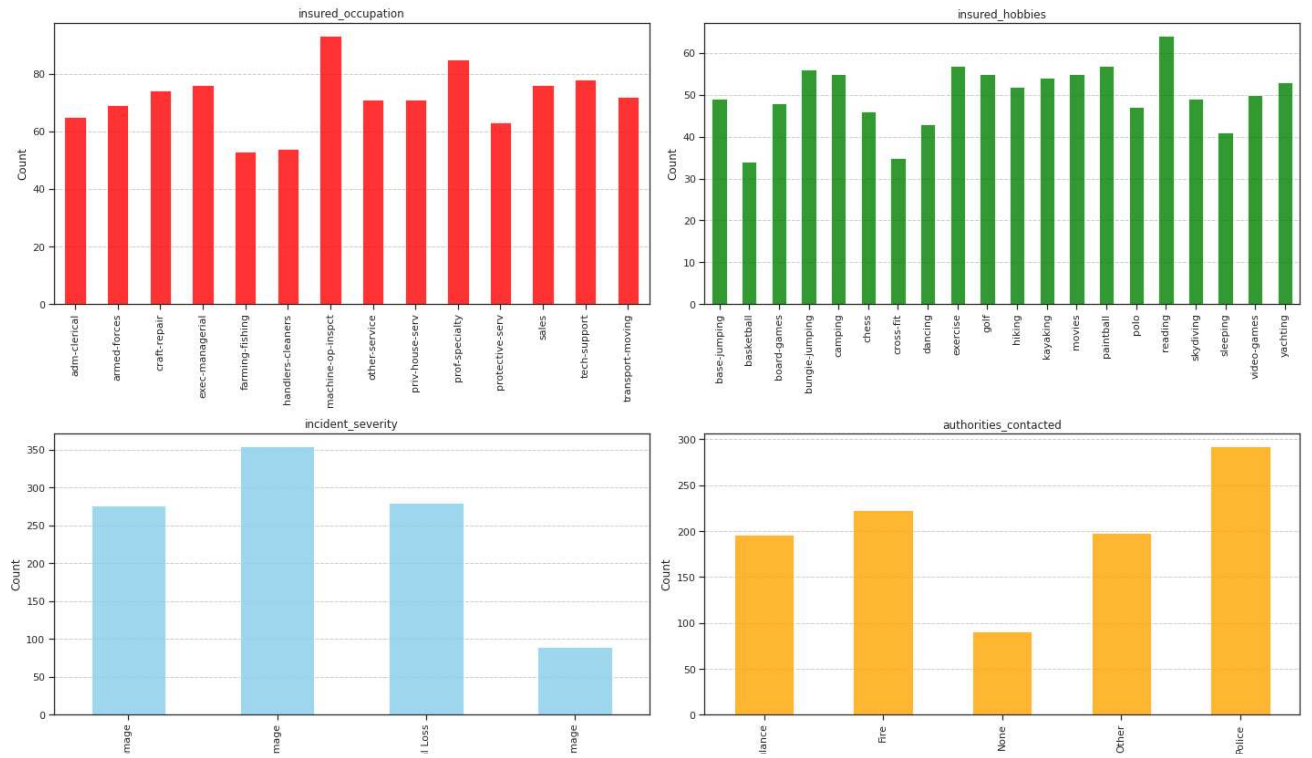
```
    data_frame[col].value_counts().sort_index().plot(kind='bar', ax=ax, color=colo
```

```
    ax.set(title=col, xlabel='', ylabel='Count')
```

```
    ax.grid(axis='y', linestyle='--')
```

```
plt.tight_layout()
```

```
plt.show()
```



Distribution of skewness by plotting numerical values

Checking how the data has been distriubted in each column

plt.figure(figsize=(25,35), facecolor='white')

```
for i, column in enumerate(nc):
```

```
    plt.subplot(6, 4, i+1)
```

```
    sns.histplot(data_frame[column], kde=True, color='darkviolet', alpha=0.7)
```

```
    plt.xlabel(column, fontsize=18)
```

```
plt.tight_layout()
```

```
plt.show()
```



we remove skewness later



Bivariate Analysis



Bivariate Analysis

Plot the data using a scatterplot to visualize relationships between variables

```
colors = {'Y': 'blue', 'N': 'orange'}
```

```
fig, axs = plt.subplots(nrows=2, ncols=2, figsize=(18, 13))
```

```
# Create the scatterplots
```

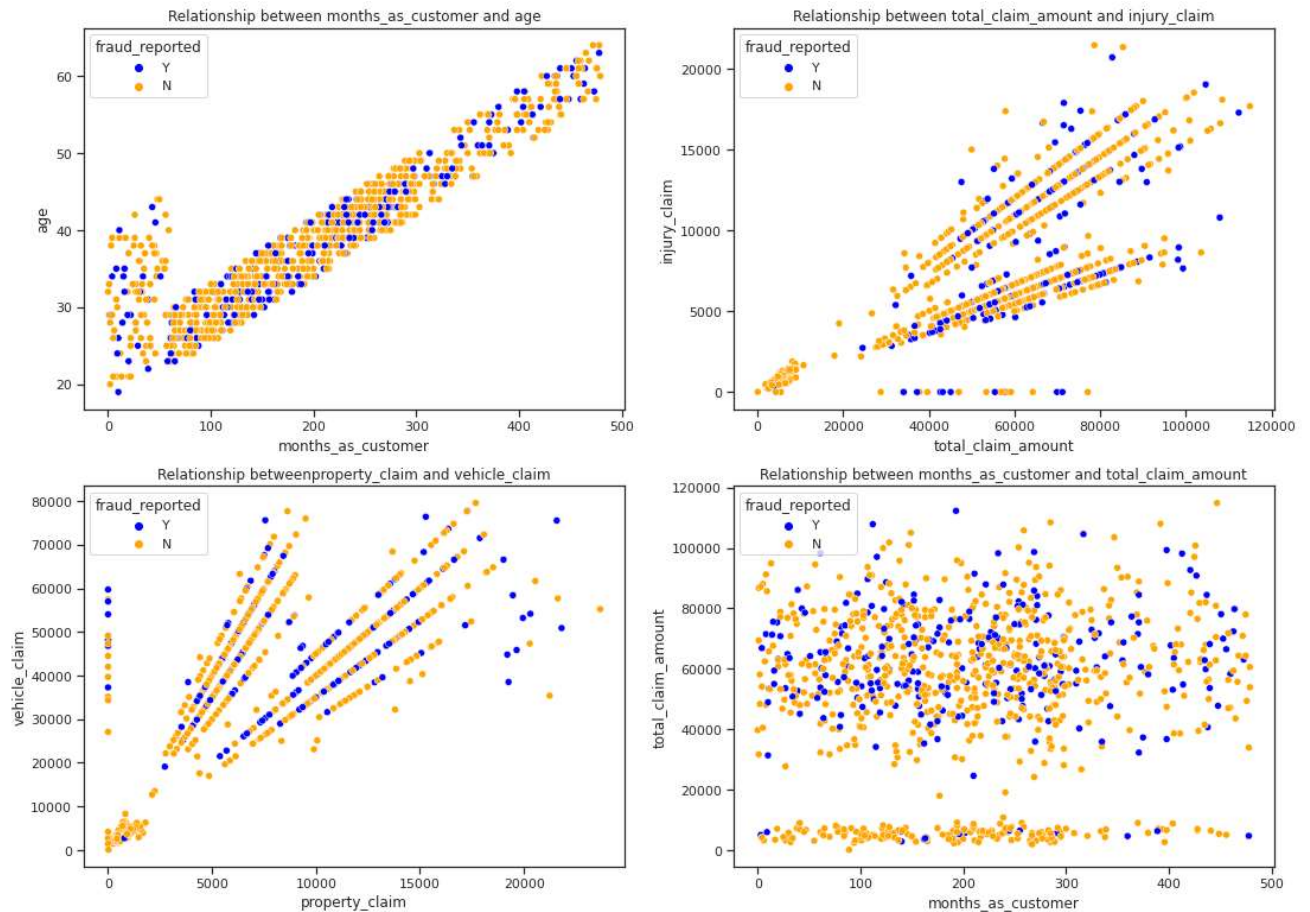
```
sns.scatterplot(x='months_as_customer', y='age', hue='fraud_reported', data=data_frame, ax=axs[0, 0]).set_title('Relationship between months_as_customer and age')
```

```
sns.scatterplot(x='total_claim_amount', y='injury_claim', hue='fraud_reported', data=data_frame, ax=axs[0, 1]).set_title('Relationship between total_claim_amount and injury_claim')
```

```
sns.scatterplot(x='property_claim', y='vehicle_claim', hue='fraud_reported', data=data_frame, ax=axs[1, 0]).set_title('Relationship between property_claim and vehicle_claim')
```

```
sns.scatterplot(x='months_as_customer', y='total_claim_amount', hue='fraud_reported', data=data_frame, ax=axs[1, 1]).set_title('Relationship between months_as_customer and total_claim_amount')
```

```
plt.show()
```



```
# Create a 2x2 grid of subplots
```

```
fig, axes = plt.subplots(nrows=2, ncols=2, figsize=(12, 10))
```

```
# Plot a violin plot comparesion
```

```
sns.violinplot(x='insured_sex', y='age', data=data_frame, hue='fraud_reported', palette='S')
axes[0, 0].set_title('\nComparison of Age by Insured Sex')
```

```
axes[0, 0].set_xlabel('Insured Sex\n')
```

```
axes[0, 0].set_ylabel('Age')
```

```
sns.violinplot(x='policy_state', y='witnesses', data=data_frame, hue='fraud_reported', pal
axes[0, 1].set_title('\nComparison of Witnesses by Policy State')
```

```
axes[0, 1].set_xlabel('Policy State\n')
```

```
axes[0, 1].set_ylabel('Witnesses')
```

```
sns.violinplot(x='accident_csl', y='property_claim', data=data_frame, hue='fraud_reported'
axes[1, 0].set_title('\nComparison of Property Claim by CSL per Accident')
```

```
axes[1, 0].set_xlabel('Accident Csl\n')
```

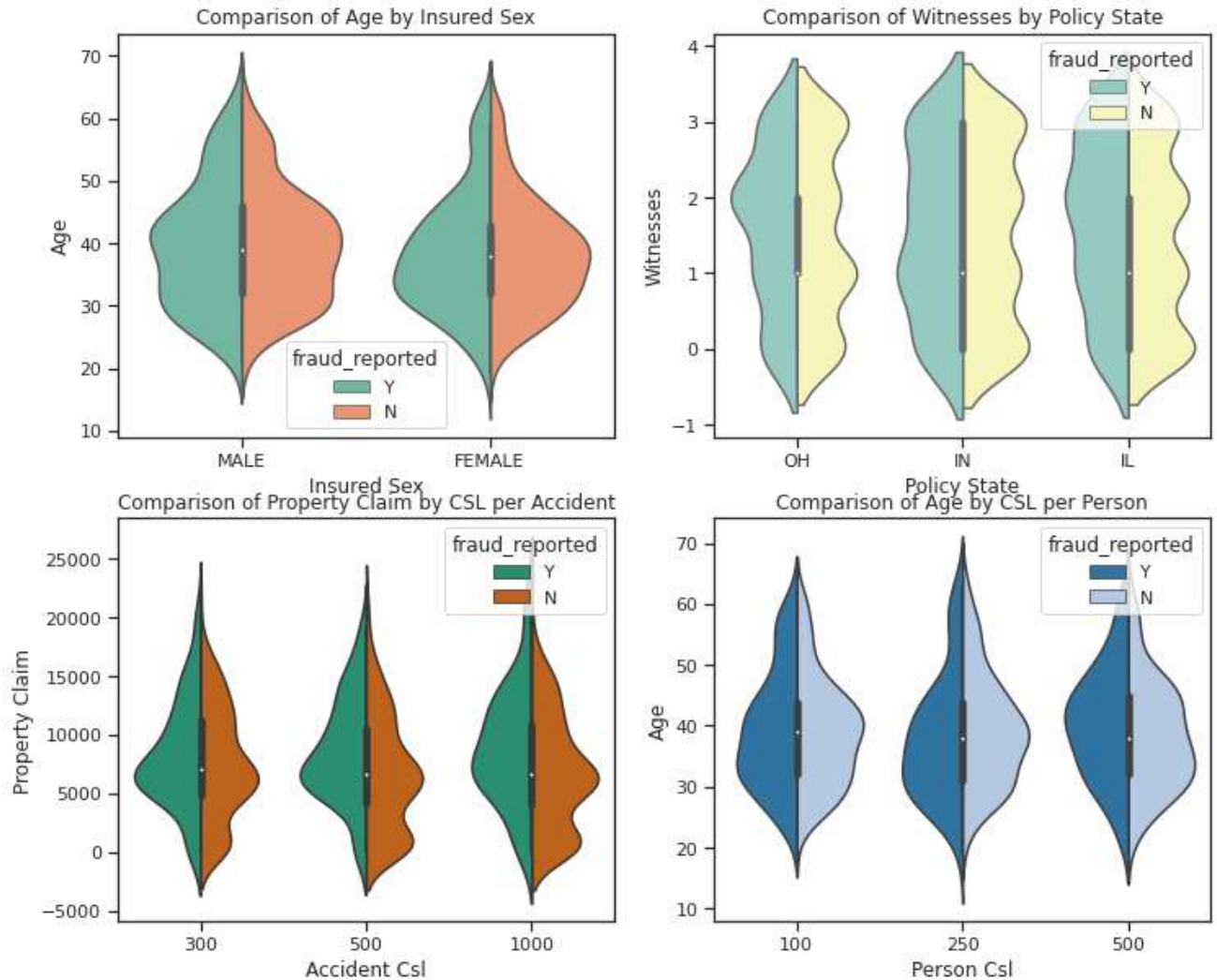
```
axes[1, 0].set_ylabel('Property Claim')
```

```
sns.violinplot(x='person_csl', y='age', data=data_frame, hue='fraud_reported', palette='ta
axes[1, 1].set_title('\nComparison of Age by CSL per Person')
```

```
axes[1, 1].set_xlabel('Person Csl\n')
```

```
axes[1, 1].set_ylabel('Age')
```

```
plt.show()
```



```
# Create a 2x2 subplot with a 12x12 figure size
fig, axes = plt.subplots(2, 2, figsize=(12, 12))
```

```
sns.violinplot(x='fraud_reported', y='total_claim_amount', ax=axes[0, 0], data=data_frame,
axes[0, 0].set_title('Total ClaimAmount vs. Fraud Reported')
axes[0, 0].set_xlabel('Fraud Reported')
axes[0, 0].set_ylabel('Total Claim Amount')
```

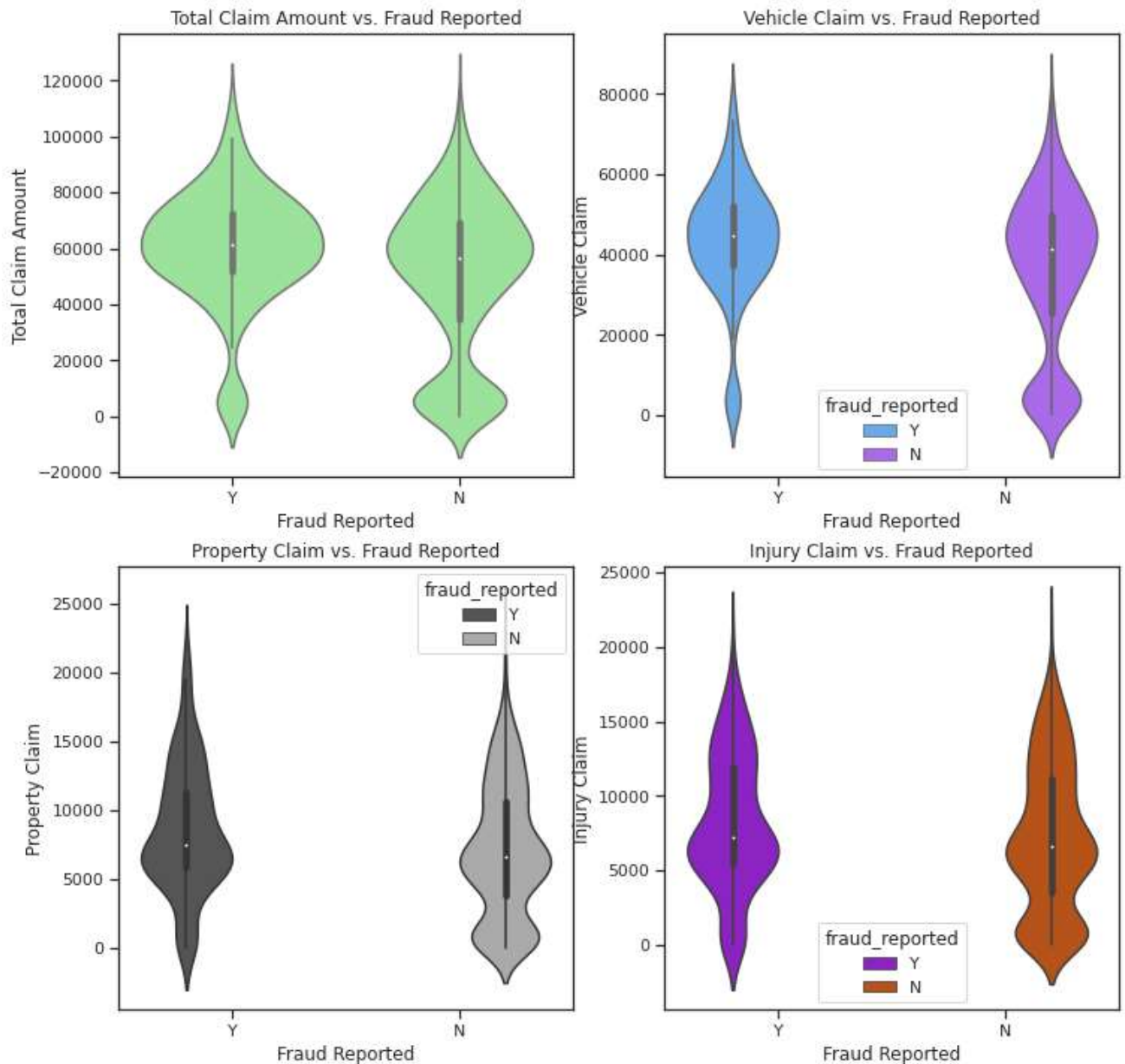
```
sns.violinplot(x='fraud_reported', y='vehicle_claim', ax=axes[0, 1], data=data_frame, hue=
axes[0, 1].set_title('Vehicle Claim vs. Fraud Reported')
axes[0, 1].set_xlabel('Fraud Reported')
axes[0, 1].set_ylabel('Vehicle Claim')
```

```
sns.violinplot(x='fraud_reported', y='property_claim', ax=axes[1, 0], data=data_frame, hue
axes[1, 0].set_title('Property Claim vs. Fraud Reported')
axes[1, 0].set_xlabel('Fraud Reported')
axes[1, 0].set_ylabel('Property Claim')
```

```
sns.violinplot(x='fraud_reported', y='injury_claim', ax=axes[1, 1], data=data_frame, hue="
axes[1, 1].set_title('Injury Claim vs. Fraud Reported')
axes[1, 1].set_xlabel('Fraud Reported')
```

```
axes[1, 1].set_ylabel('Injury Claim')
```

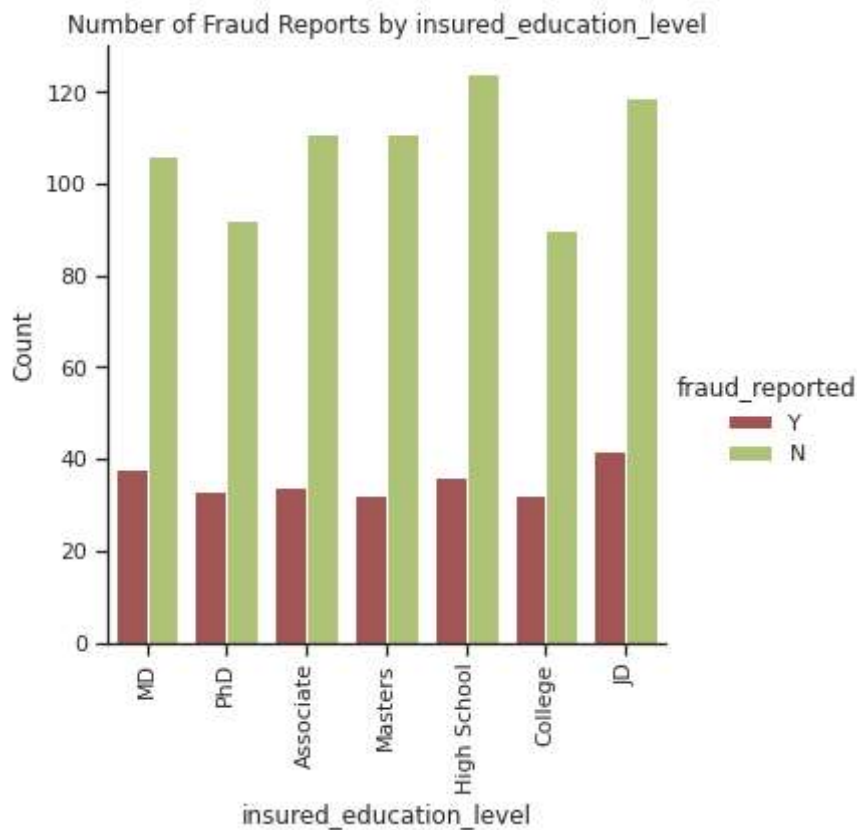
```
plt.show()
```



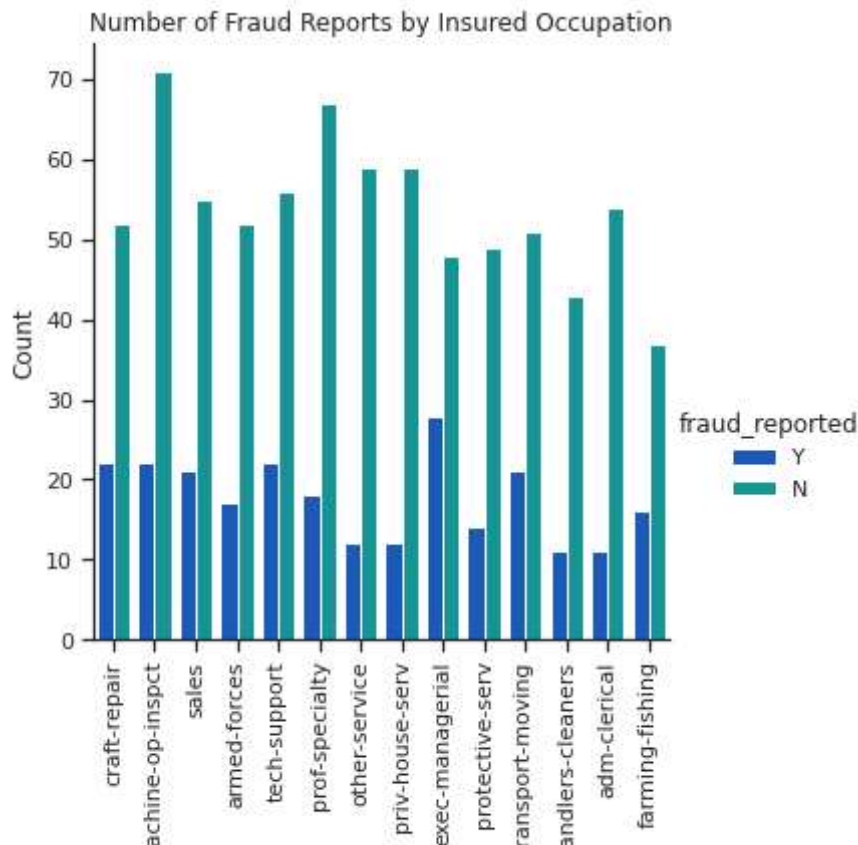
```
#Comparing policy_state and fraud_reported
sns.catplot(x='policy_state', kind='count', data=data_frame, hue='fraud_reported', palette
plt.title('Number of policy_state by Fraud Reports')
plt.xlabel('Insured Occupation')
plt.ylabel('Count')
plt.xticks(rotation=90)
plt.show()
```



```
#Comparing insured_education_level and fraud_reported
sns.catplot(x='insured_education_level', kind='count', data=data_frame, hue='fraud_reported')
plt.title('Number of Fraud Reports by insured_education_level')
plt.xlabel('insured_education_level')
plt.ylabel('Count')
plt.xticks(rotation=90)
plt.show()
```



```
#Comparing insured_occupation and fraud_reported
sns.catplot(x='insured_occupation', kind='count', data=data_frame, hue='fraud_reported', p
plt.title('Number of Fraud Reports by Insured Occupation')
plt.xlabel('Insured Occupation')
plt.ylabel('Count')
plt.xticks(rotation=90)
plt.show()
```



```
# Create a factor plot of insured hobbies vs. fraud reported
sns.catplot(x='insured_hobbies', kind='count', data=data_frame, hue='fraud_reported', palette='magma')
plt.title('Count of Insured Hobbies by Fraud Reported')
plt.xlabel('Insured Hobbies')
plt.ylabel('Count')
plt.xticks(rotation=90)

# Display the plot
plt.show()
```



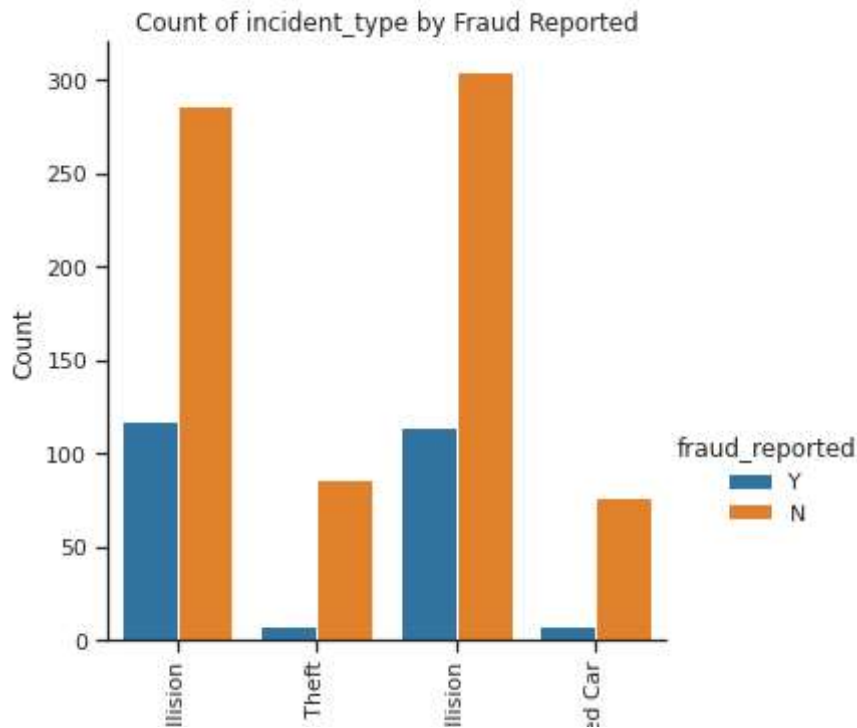
```
# Create a factor plot of insured_relationship and fraud_reported
sns.catplot(x='insured_relationship', kind='count', data=data_frame, hue='fraud_reported',
plt.title('Count of insured_relationship by Fraud Reported')
plt.xlabel('fraud_reported')
plt.ylabel('Count')
plt.xticks(rotation=90)

# Display the plot
plt.show()
```



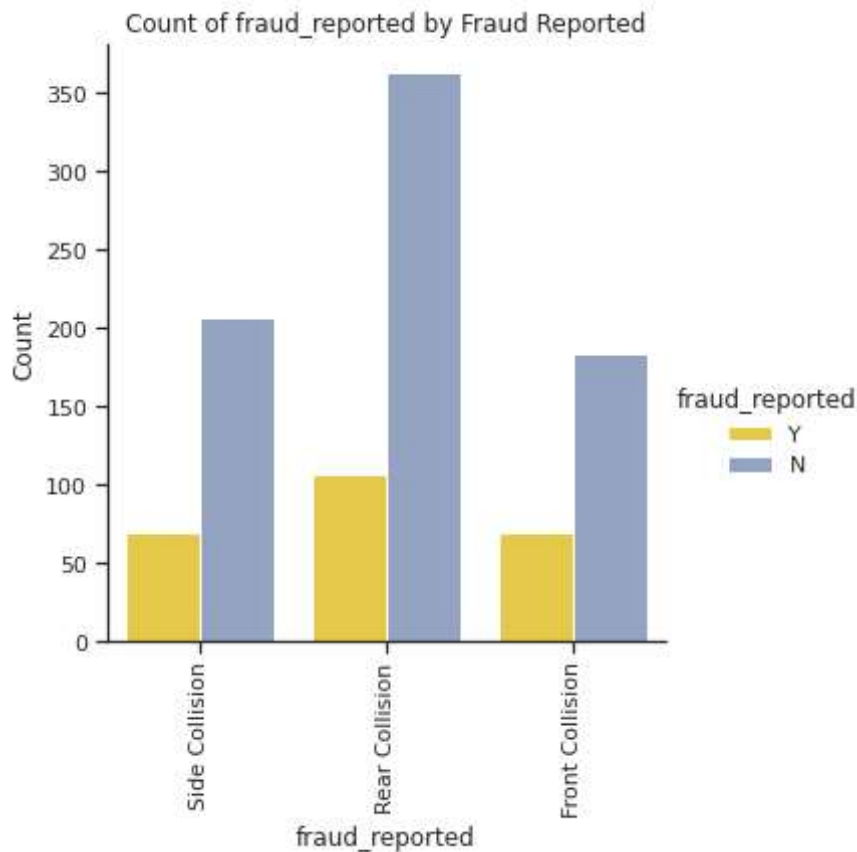
```
# Create a factor plot of incident_type and fraud_reported
sns.catplot(x='incident_type', kind='count', data=data_frame, hue='fraud_reported', palette=
plt.title('Count of incident_type by Fraud Reported')
plt.xlabel('incident_type')
plt.ylabel('Count')
plt.xticks(rotation=90)

# Display the plot
plt.show()
```



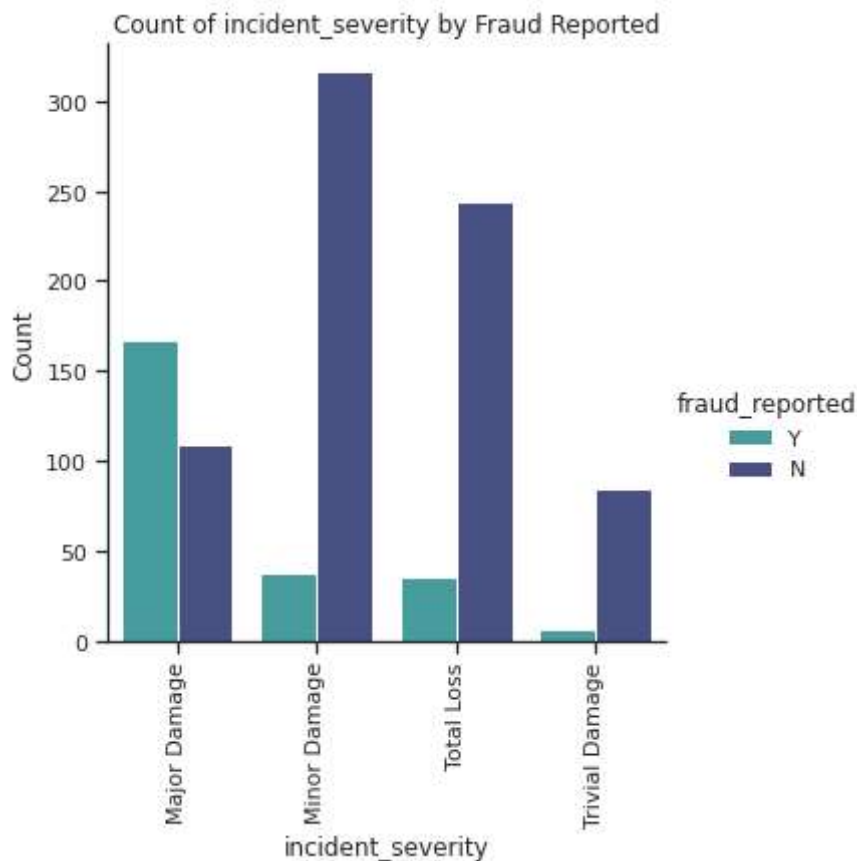
```
# Create a factor plot of collision_type and fraud_reported
sns.catplot(x='collision_type', kind='count', data=data_frame, hue='fraud_reported', palette='magma')
plt.title('Count of fraud_reported by Fraud Reported')
plt.xlabel('collision_type')
plt.ylabel('Count')
plt.xticks(rotation=90)
```

```
# Display the plot
plt.show()
```



```
# Create a factor plot of incident_severity and fraud_reported
sns.catplot(x='incident_severity', kind='count', data=data_frame, hue='fraud_reported', pa
plt.title('Count of incident_severity by Fraud Reported')
plt.xlabel('incident_severity')
plt.ylabel('Count')
plt.xticks(rotation=90)

# Display the plot
plt.show()
```



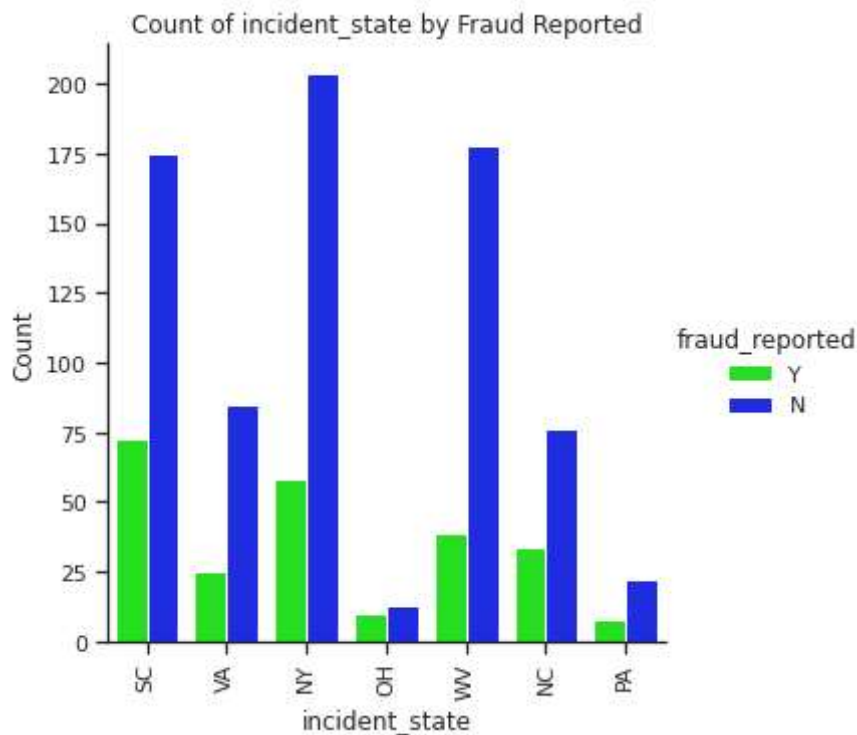
```
# Create a factor plot of authorities_contacted and fraud_reported
sns.catplot(x='authorities_contacted', kind='count', data=data_frame, hue='fraud_reported'
plt.title('Count of authorities_contacted by Fraud Reported')
plt.xlabel('authorities_contacted')
plt.ylabel('Count')
plt.xticks(rotation=90)

# Display the plot
plt.show()
```



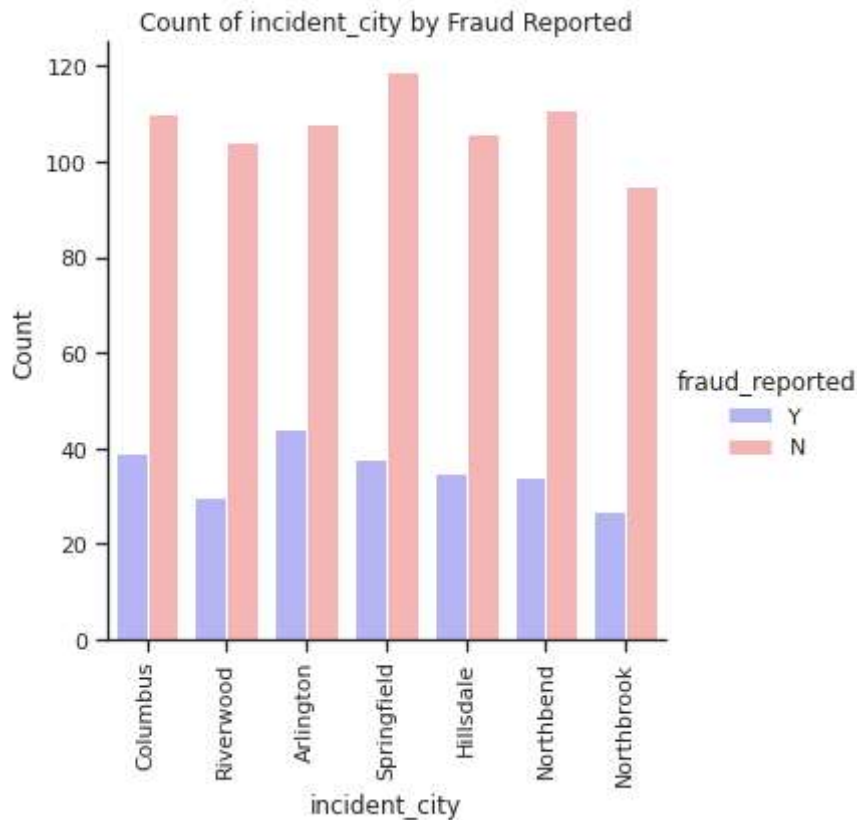
```
# Create a factor plot of incident_state and fraud_reported
sns.catplot(x='incident_state', kind='count', data=data_frame, hue='fraud_reported')
plt.title('Count of incident_state by Fraud Reported')
plt.xlabel('incident_state')
plt.ylabel('Count')
plt.xticks(rotation=90)
```

```
# Display the plot
plt.show()
```



```
# Create a factor plot of incident_city and fraud_reported
sns.catplot(x='incident_city', kind='count', data=data_frame, hue='fraud_reported', palette=
plt.title('Count of incident_city by Fraud Reported')
plt.xlabel('incident_city')
plt.ylabel('Count')
plt.xticks(rotation=90)
```

```
# Display the plot
plt.show()
```



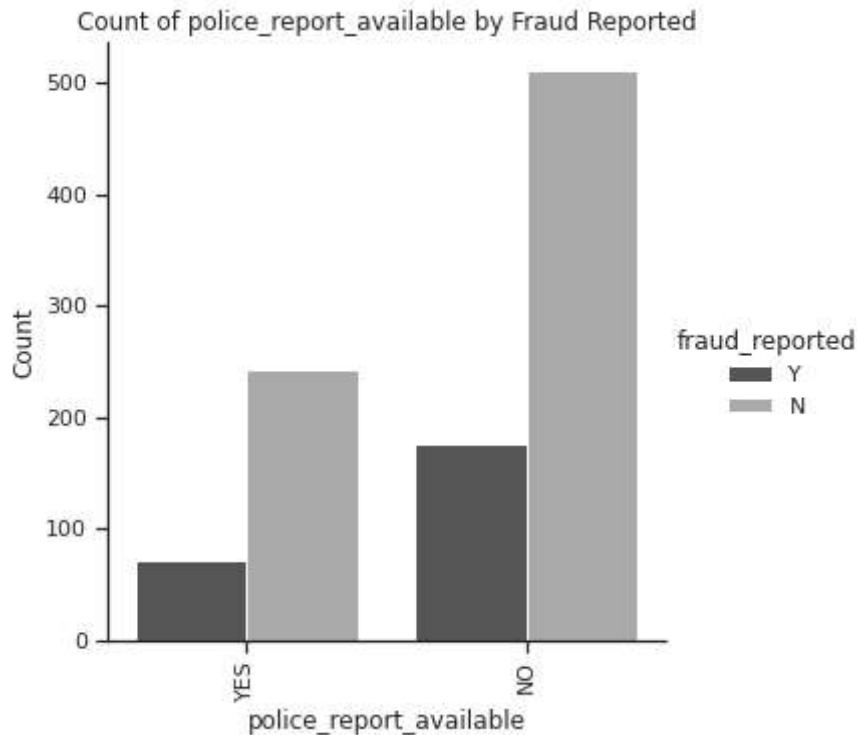
```
# Create a factor plot of property_damage and fraud_reported
sns.catplot(x='property_damage', kind='count', data=data_frame, hue='fraud_reported', pale
plt.title('Count of property_damage by Fraud Reported')
plt.xlabel('property_damage')
plt.ylabel('Count')
plt.xticks(rotation=90)

# Display the plot
plt.show()
```



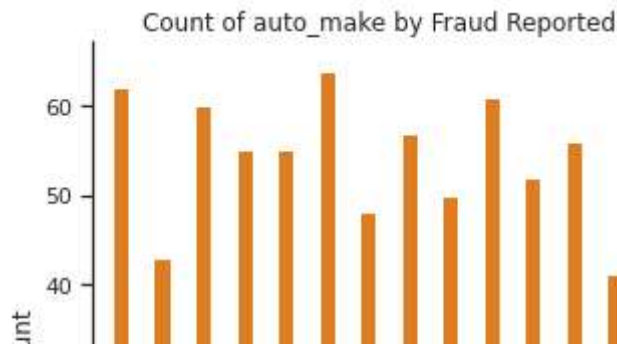
```
# Create a factor plot of police_report_available and fraud_reported
sns.catplot(x='police_report_available', kind='count', data=data_frame, hue='fraud_reported')
plt.title('Count of police_report_available by Fraud Reported')
plt.xlabel('police_report_available')
plt.ylabel('Count')
plt.xticks(rotation=90)

# Display the plot
plt.show()
```



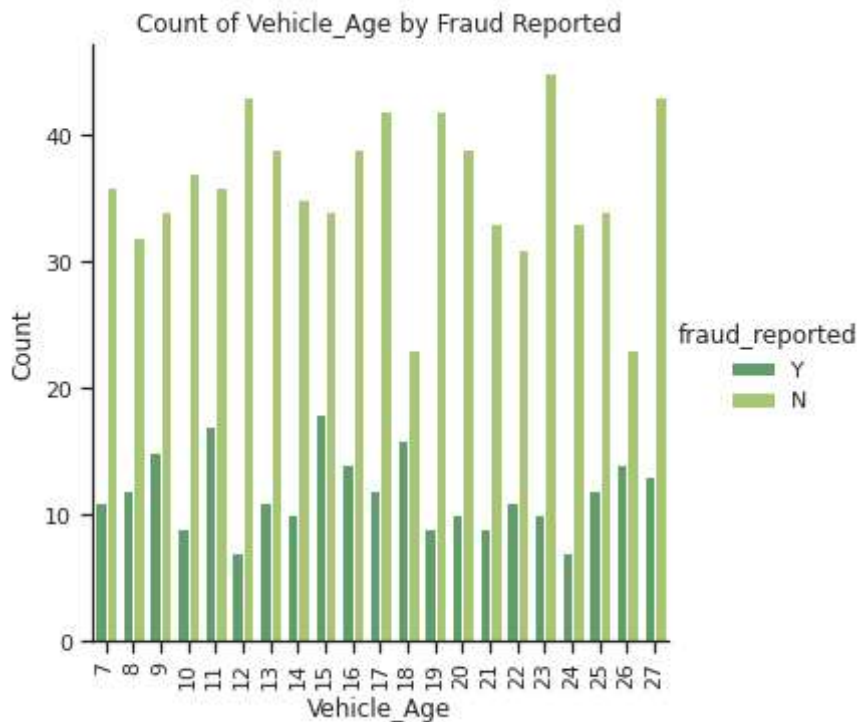
```
# Create a factor plot of auto_make and fraud_reported
sns.catplot(x='auto_make', kind='count', data=data_frame, hue='fraud_reported', palette="b")
plt.title('Count of auto_make by Fraud Reported')
plt.xlabel('auto_make')
plt.ylabel('Count')
plt.xticks(rotation=90)

# Display the plot
plt.show()
```



```
# Create a factor plot of Vehicle_Age and fraud_reported
sns.catplot(x='Vehicle_Age', kind='count', data=data_frame, hue='fraud_reported', palette=
plt.title('Count of Vehicle_Age by Fraud Reported')
plt.xlabel('Vehicle_Age')
plt.ylabel('Count')
plt.xticks(rotation=90)

# Display the plot
plt.show()
```



```
# Multivariate Analysis: Pairwise Relationships with Fraud Reported
sns.set(style="ticks", color_codes=True)
sns.pairplot(data_frame, hue="fraud_reported", palette="muted")

plt.suptitle('Pairwise Relationships with Fraud Reported')
plt.subplots_adjust(top=0.95)

plt.show()
```



```
# Identifying the outliers
# Visualizing Outliers with Box Plots
fig, axs = plt.subplots(nrows=6, ncols=4, figsize=(25, 35), facecolor='white')

for i, column in enumerate(nc):
    if i < 23:
        ax = axs.flatten()[i]
        sns.boxplot(data_frame[column], color="lightcoral", ax=ax)
        ax.set_xlabel(column, fontsize=20)

plt.tight_layout()
```



```
# Feature containing outliers
```

```
features = data_frame[['age','policy_annual_premium','total_claim_amount','property_claim']
```

```
# Removing outliers using z-score method
```

```
z_scores = zscore(features)
```

```
abs_z_scores = np.abs(z_scores)
```

```
# Creating a new dataframe with filtered entries
```

```
filtered_entries = (abs_z_scores < 3).all(axis=1)
```

```
new_data_frame = data_frame[filtered_entries]
```

```
# cheaking shapes of datasets
```

```
print("Shape of original dataset ",data_frame.shape)
```

```
print("Shape of new dataset", new_data_frame.shape)
```

```
# Calculate the loss percentage
```

```
loss_percent = ((1000 - 996) / 1000) * 100
```

```
print(f"loss percentage of dataset is: {loss_percent:.2f}%")
```

```

Shape of original dataset (1000, 39)
Shape of new dataset (996, 39)
loss percentage of dataset is: 0.40%

```

```

# Remove outliers using the IQR method
# Calculate the 1st quantile
Q1 = features.quantile(0.25)
# Calculate the 3rd quantile
Q3 = features.quantile(0.75)
# Calculate the interquartile range (IQR)
IQR = Q3 - Q1
outlier_threshold = 1.5
lower_bound = Q1 - outlier_threshold * IQR
upper_bound = Q3 + outlier_threshold * IQR
is_outlier = (data_frame < lower_bound) | (data_frame > upper_bound)
new_df_irq = data_frame[~is_outlier.any(axis=1)]
print("Shape of original dataset ", data_frame.shape)
print("Shape of new dataset", new_df_irq.shape)
# Calculate the loss percentage
loss_percent = ((1000 - 942) / 1000) * 100
print(f"loss percentage of dataset is: {loss_percent:.2f}%")

```

```

Shape of original dataset (1000, 39)
Shape of new dataset (942, 39)
loss percentage of dataset is: 5.80%

```

We compared the effectiveness of two methods, IQR and Zscore, to remove outliers from our dataset. Although the IQR method had lower data loss than Zscore, we ultimately decided to use Zscore as it provided better results.

```

# Calculate the skewness of each feature in the new_df dataframe
skewness = new_data_frame.skew()
# Sort the skewness values in ascending order
skewness_sorted = skewness.sort_values()
# Print the sorted skewness values
print(skewness_sorted)

```

```

vehicle_claim          -0.619755
total_claim_amount     -0.593473
capital-loss           -0.393015
incident_hour_of_the_day -0.039123
policy_bind_Month      -0.029722
bodily_injuries         0.011117
witnesses              0.025758
policy_bind_Day         0.028923
policy_annual_premium   0.032042
Vehicle_Age            0.049276
incident_Day           0.055659
policy_bind_Year        0.058499
injury_claim           0.267970
property_claim          0.357130
months_as_customer      0.359605

```

```

person_csl          0.413713
policy_deductable   0.473229
age                 0.474526
capital-gains       0.478850
number_of_vehicles_involved 0.500364
accident_csl        0.609316
incident_Month      1.377097
dtype: float64

```

```

# Remove skewness using the Yeo-Johnson method to improve prediction accuracy
# Define a list of features with skewness
skew_features = ["total_claim_amount", "vehicle_claim", "incident_Month", "accident_csl"]

# Apply the Yeo-Johnson transformation to the skew features
transf = PowerTransformer(method='yeo-johnson')
new_data_frame[skew_features] = transf.fit_transform(new_data_frame[skew_features].values)

# Print the first few rows of the transformed features
print(new_data_frame[skew_features].head())

```

```

      total_claim_amount  vehicle_claim  incident_Month  accident_csl
0          0.717556         0.754553        -1.101370        0.052612
1         -1.777785        -1.787353        -1.101370        0.052612
2         -0.716483        -0.820820        -0.026479       -1.174021
3          0.392931         0.678427         1.553647        0.052612
4         -1.730555        -1.740710        -0.026479        1.313327

```

```

# Check skewness after applying the Yeo-Johnson transformation
# Calculate the skewness of the transformed features
skewness = new_data_frame[skew_features].skew()
print(skewness)

```

```

total_claim_amount    -0.508953
vehicle_claim          -0.521354
incident_Month         0.305741
accident_csl           0.110964
dtype: float64

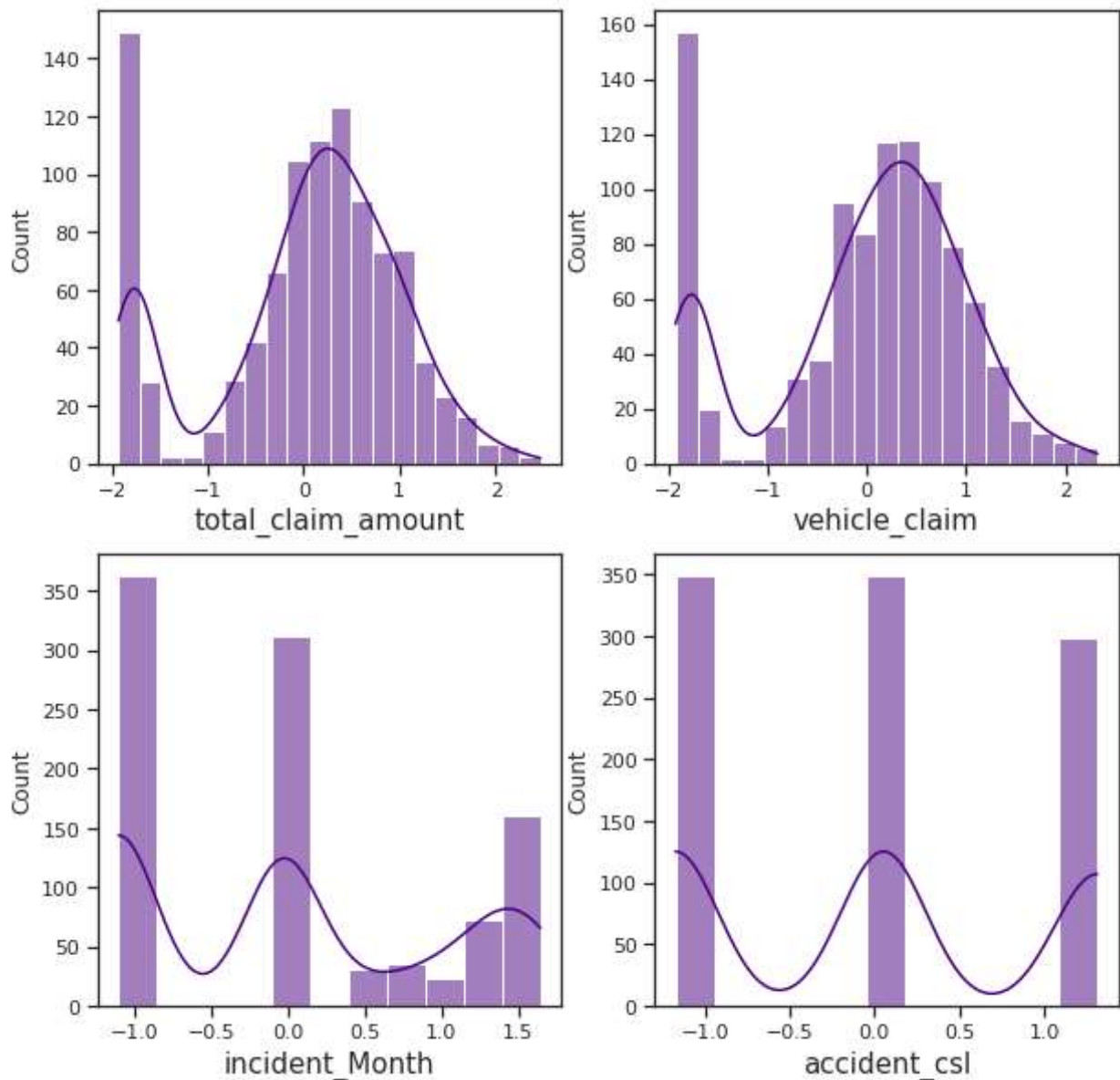
```

```

# After removing skewness using yeo-johnson method, checkind how the data has been distrib
skew = ["total_claim_amount", "vehicle_claim", "incident_Month", "accident_csl"]
plt.figure(figsize=(10,10), facecolor='white')
# Set the plot number to 1
plotnumber = 1
# Loop through each column in the list of skewed columns
for column in new_data_frame[skew]:
    if plotnumber<=4:
        # Createing a subplot
        ax = plt.subplot(2,2,plotnumber)
        sns.histplot(new_data_frame[column],color='indigo',kde=True)
        plt.xlabel(column,fontsize=15)
        plotnumber+=1

```

```
plt.show()
```



```
# Applying label encoding to categorical columns.  
# convert categorical column into numerical column  
LE=LabelEncoder()  
new_data_frame[cc]= new_data_frame[cc].apply(LE.fit_transform)  
  
new_data_frame[cc].head()
```

policy_state insured_sex insured_education_level insured_occupation insured_h

```
# check the correlation between the target variable and features variables using HEAT map
# Calculate the correlation matrix
correlation_matrix = new_data_frame.corr()

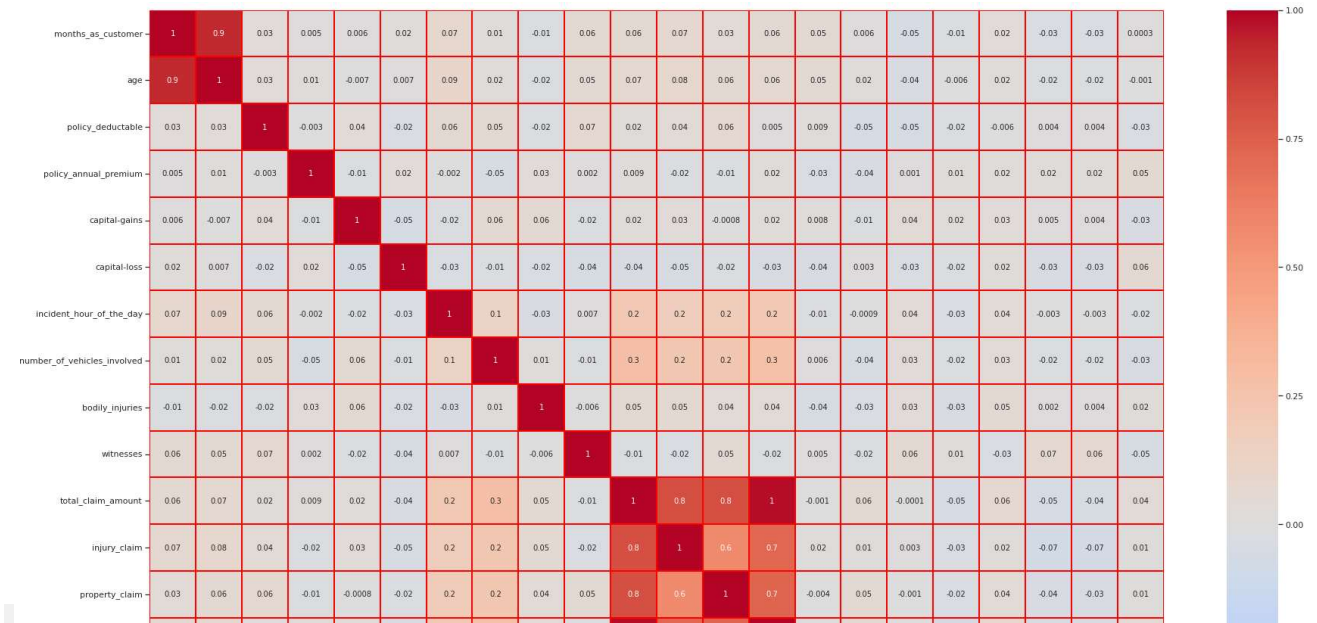
# Print the correlation matrix
correlation_matrix
```

	months_as_customer	age	policy_state	policy_de
months_as_customer	1.000000	0.922092	0.000118	
age	0.922092	1.000000	-0.015182	
policy_state	0.000118	-0.015182	1.000000	
policy_deductable	0.023512	0.026772	0.010740	
policy_annual_premium	-0.003378	0.005890	0.014475	
insured_sex	0.059002	0.072900	-0.019294	
insured_education_level	-0.000848	0.001485	-0.032551	
insured_occupation	0.005365	0.014030	-0.031016	
insured_hobbies	-0.092575	-0.080587	-0.040215	
insured_relationship	0.071097	0.075940	-0.018855	
capital-gains	0.007601	-0.006333	0.072552	
capital-loss	0.020771	0.008192	-0.032268	
incident_type	-0.017202	-0.025979	0.031769	
collision_type	-0.072479	-0.075892	0.050953	
incident_severity	-0.061932	-0.060361	-0.026728	

```
# Visualizing the correlation matrix
corr_matrix = data_frame.corr()
# Set the figure size and plot the heatmap
plt.figure(figsize=(30,25))
sns.heatmap(corr_matrix, linewidths=.1, vmin=-1, vmax=1, fmt='.1g', linecolor="red")

# Rotate the y-axis tick labels to horizontal
plt.yticks(rotation=0)

# Show the plot
plt.show()
```

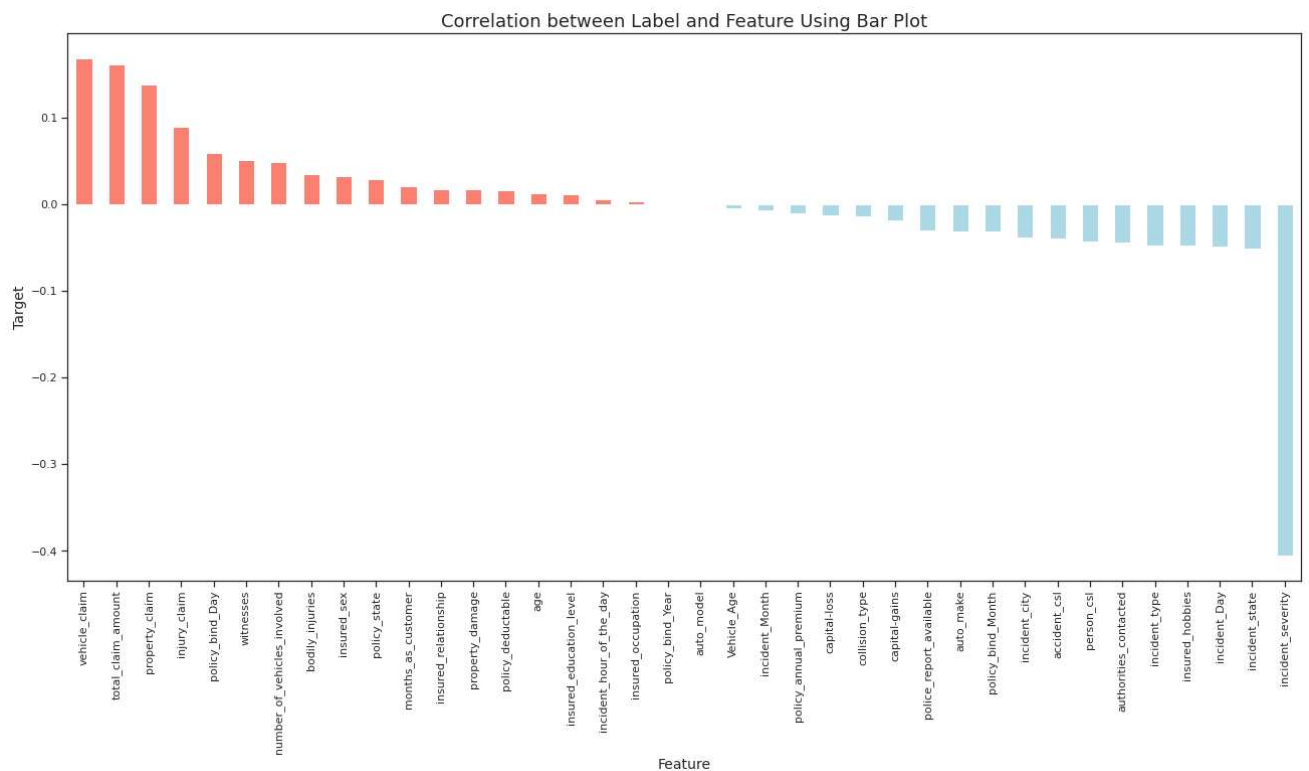


```
# # Creating a visualization to show the correlation between the target variable and the f
corr = new_data_frame.corr()['fraud_reported'].sort_values(ascending=False).drop(['fraud_r
```

```
# Set the figure size and plot the bar chart
plt.figure(figsize=(22,10))
corr.plot(kind='bar', color=['lightblue' if c < 0 else 'salmon' for c in corr])
```

```
# Set the x- and y-axis labels and title
plt.xlabel('Feature',fontsize=14)
plt.ylabel('Target',fontsize=14)
plt.title('Correlation between Label and Feature Using Bar Plot',fontsize=18)
```

```
# Show the plot
plt.show()
```



```
# Split the dataset into features and target
X = new_data_frame.drop("fraud_reported", axis=1)
Y = new_data_frame["fraud_reported"]
```

```
# Print the dimensions of X and y
print("Dimension of X:", X.shape)
print("Dimension of y:", Y.shape)
```

```
Dimension of X: (996, 38)
```

```
Dimension of y: (996,)
```

```
# Perform feature scaling using StandardScaler
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
X = pd.DataFrame(X_scaled, columns=X.columns)
```

```
# Print the first few rows of X after scaling
print(X.head())
```

```
   months_as_customer  age  policy_state  policy_deductable \
0          1.074671  0.987190      1.186130      -0.224722
1          0.204846  0.330455     -0.018137       1.409024
2         -0.612790 -1.092470      1.186130       1.409024
3          0.448397  0.221000     -1.222403       1.409024
4          0.204846  0.549367     -1.222403     -0.224722

   policy_annual_premium  insured_sex  insured_education_level \
0          0.621371      1.075102          0.531088
1         -0.251375      1.075102          0.531088
2          0.647301     -0.930144          1.557206
3          0.658123     -0.930144          1.557206
4          1.358059      1.075102         -1.521148

   insured_occupation  insured_hobbies  insured_relationship  ...  auto_make \
0         -1.162296      1.280299      -1.436113  ...    0.857248
1         -0.166257      0.928186      -0.246617  ...    0.360986
2          1.078792     -1.360550       0.348131  ...   -0.631538
3         -1.411305     -1.360550       0.942878  ...   -0.879669
4          1.078792     -1.360550       0.942878  ...  -1.624063

   auto_model  policy_bind_Day  policy_bind_Month  policy_bind_Year \
0    -1.644810      0.177487      1.013751      1.686176
1    -0.657437      1.309223     -0.166145      0.599693
2     0.958263     -0.727901     -0.166145     -0.215170
3     1.317308      1.082875     -0.461119     -1.573274
4     1.048024     -1.067422     -0.166145      1.686176

   incident_Day  incident_Month  person_csl  accident_csl  Vehicle_Age
0     1.139807     -1.101370     -0.137684     0.052612     0.182338
1     0.756374     -1.101370     -0.137684     0.052612    -0.316587
2     0.852233     -0.026479    -1.066352    -1.174021    -0.316587
3    -1.160788      1.553647     -0.137684     0.052612    -1.480744
4     0.372942     -0.026479     1.410096     1.313327    -0.649203
```

```
[5 rows x 38 columns]
```

```
# Define a function to calculate VIF values for a DataFrame
def v_i_f(data_frame):
    variance_data = pd.DataFrame()
    variance_data["Feature"] = data_frame.columns
    variance_data["VIF"] = [variance_inflation_factor(data_frame.values, i) for i in range
    return variance_data

# Calculate the initial VIF values for the feature DataFrame x
data_vif = v_i_f(X)
print("Initial VIF values:\n", data_vif)
print("\n_____ \n")
# Remove the columns with high VIF values one by one and recalculate VIF values
to_drop = ["total_claim_amount", "accident_csl"]
for col in to_drop:
    X = X.drop(columns=col)
    data_vif = v_i_f(X)
    print("\n_____ \n")
    print("\nAfter dropping", col, "VIF values:\n", data_vif)
    print("\n_____ \n")

# Print the value counts of the target variable y
print("Value counts of y:\n", Y.value_counts())
```

```

19 number_of_vehicles_involved 5.110588
20      property_damage      1.030105
21      bodily_injuries      1.026183
22      witnesses          1.043608
23      police_report_available 1.038953
24      injury_claim        2.175649
25      property_claim      2.253397
26      vehicle_claim       3.359149
27      auto_make          1.073774
28      auto_model         1.065523
29      policy_bind_Day     1.024664
30      policy_bind_Month   1.039515
31      policy_bind_Year    1.028024
32      incident_Day        1.967723
33      incident_Month      1.971186
34      person_csl         1.032569
35      Vehicle_Age         1.040165

```

Value counts of y:

0 750

1 246

Name: fraud_reported, dtype: int64

```

# Import the SMOTE algorithm from the imblearn library
from imblearn.over_sampling import SMOTE

```

```

# Instantiate the SMOTE object
smote = SMOTE()

```

```

# Use the fit_resample method to oversample the data
# x: the feature data to be oversampled
# y: the corresponding label data
x_resampled, y_resampled = smote.fit_resample(X, Y)

```

```

# Check the value count of the target column

```

```
target_counts = Y.value_counts()
```

```
print("target counts \n " , target_counts)
```

```
# Visualize the data after oversampling
```

```
print("\n_____ \n")
```

```
sns.countplot(x=y_resampled, palette='tab10')
```

```
plt.title("Balance data after OverSampling")
```

```
plt.show()
```

```
target counts
0      750
1      246
Name: fraud_reported, dtype: int64
```



```
from imblearn.under_sampling import RandomUnderSampler
```

```
# Instantiate RandomUnderSampler with desired sampling strategy
undersampler = RandomUnderSampler(sampling_strategy='majority')
```

```
# Resample the data
```

```
x_resampled, y_resampled = undersampler.fit_resample(X, Y)
```

```
# Check the value count of the target column
```

```
target_counts = y_resampled.value_counts()
```

```
print("target counts \n " , target_counts)
```

```
# Visualize the data after oversampling
```

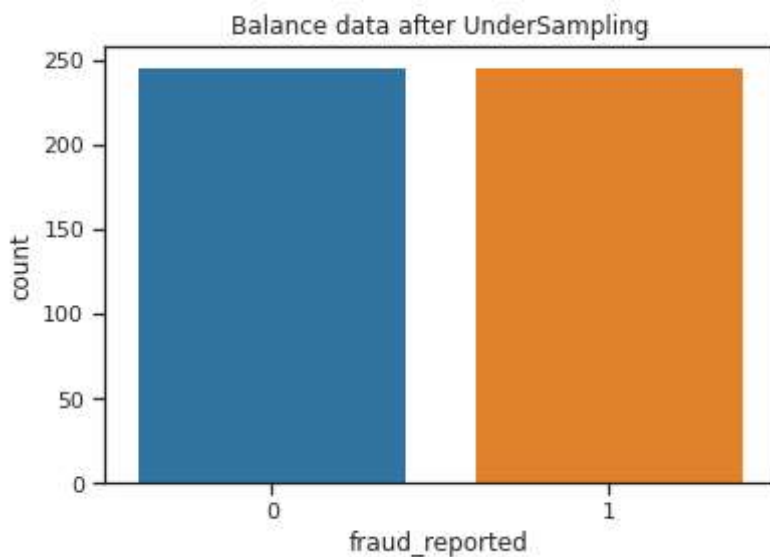
```
print("\n_____ \n")
```

```
sns.countplot(x=y_resampled, palette='tab10')
```

```
plt.title("Balance data after UnderSampling")
```

```
plt.show()
```

```
target counts
0      246
1      246
Name: fraud_reported, dtype: int64
```



```

from imblearn.over_sampling import SMOTE
from imblearn.under_sampling import RandomUnderSampler
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

# Split the data into training and testing sets
x_train, x_test, y_train, y_test = train_test_split(X, Y, test_size=0.2, random_state=42)

# Oversample the training data using SMOTE
smote = SMOTE()
x_train_oversampled, y_train_oversampled = smote.fit_resample(x_train, y_train)

# Undersample the training data using RandomUnderSampler
undersampler = RandomUnderSampler(sampling_strategy='majority')
x_train_undersampled, y_train_undersampled = undersampler.fit_resample(x_train, y_train)

# Train a logistic regression model on the original data
lr = LogisticRegression(random_state=42)
lr.fit(x_train, y_train)
y_pred_orig = lr.predict(x_test)

# Train a logistic regression model on the oversampled data
lr_oversampled = LogisticRegression(random_state=42)
lr_oversampled.fit(x_train_oversampled, y_train_oversampled)
y_pred_oversampled = lr_oversampled.predict(x_test)

# Train a logistic regression model on the undersampled data
lr_undersampled = LogisticRegression(random_state=42)
lr_undersampled.fit(x_train_undersampled, y_train_undersampled)
y_pred_undersampled = lr_undersampled.predict(x_test)

# Compare the accuracy of the three models
acc_orig = accuracy_score(y_test, y_pred_orig)
acc_oversampled = accuracy_score(y_test, y_pred_oversampled)
acc_undersampled = accuracy_score(y_test, y_pred_undersampled)

print(f"Accuracy of original data: {acc_orig}")
print(f"Accuracy of oversampled data: {acc_oversampled}")
print(f"Accuracy of undersampled data: {acc_undersampled}")

Accuracy of original data: 0.74
Accuracy of oversampled data: 0.73
Accuracy of undersampled data: 0.72

```

In this example code, we use SMOTE to oversample the data and RandomUnderSampler to undersample the data. We then train six classifiers (Logistic Regression, SVM, Decision Tree, KNeighbors, Random Forest, and Extra Trees) on the original, oversampled, and undersampled versions of the training data. We compute the accuracy of each model and plot the confusion matrices for each model.

To determine which version of the training data is best for each classifier, we compare the accuracy of the model trained on the original data, the model trained on the oversampled data, and the model trained on the undersampled data. We also visualize the confusion matrices to see which models are performing well and which models are making more errors.

Note that the code above assumes that the variables `x` and `y` have already been defined and contain the feature and target data, respectively. Additionally, you may need to import additional libraries depending on your specific use case.

```
from imblearn.over_sampling import SMOTE
from imblearn.under_sampling import RandomUnderSampler
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import RandomForestClassifier, ExtraTreesClassifier
from sklearn.metrics import classification_report, accuracy_score, confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

# Split the data into training and testing sets
x_train, x_test, y_train, y_test = train_test_split(X, Y, test_size=0.2, random_state=42)

# Oversample the training data using SMOTE
smote = SMOTE()
x_train_oversampled, y_train_oversampled = smote.fit_resample(x_train, y_train)

# Undersample the training data using RandomUnderSampler
undersampler = RandomUnderSampler(sampling_strategy='majority')
x_train_undersampled, y_train_undersampled = undersampler.fit_resample(x_train, y_train)

# Train multiple classifiers on the different versions of the training data
classifiers = {
    "Logistic Regression": LogisticRegression(random_state=42),
    "SVM": SVC(random_state=42),
    "Decision Tree": DecisionTreeClassifier(random_state=42),
    "KNeighbors": KNeighborsClassifier(),
    "Random Forest": RandomForestClassifier(random_state=42),
    "Extra Trees": ExtraTreesClassifier(random_state=42)
}

for name, clf in classifiers.items():
    # Train the model on the original data
    clf.fit(x_train, y_train)
    y_pred_orig = clf.predict(x_test)

    # Train the model on the oversampled data
    clf.fit(x_train_oversampled, y_train_oversampled)
    y_pred_oversampled = clf.predict(x_test)

    # Train the model on the undersampled data
    clf.fit(x_train_undersampled, y_train_undersampled)
```

```

y_pred_undersampled = clf.predict(x_test)

# Compute the accuracy of each model
acc_orig = accuracy_score(y_test, y_pred_orig)
acc_oversampled = accuracy_score(y_test, y_pred_oversampled)
acc_undersampled = accuracy_score(y_test, y_pred_undersampled)
print("\n-----\n")
print(f"                                {name}:")
print(f"Accuracy of original data: {acc_orig:.3f}")
print(f"::::::::::::Classification Report of original data::::::::::::\n{classification_report(y_test, y_pred_orig, target_names=class_names)}")
print("\n-----\n")
print(f"Accuracy of oversampled data: {acc_oversampled:.3f}")
print(f"::::::::::::Classification Report of oversampled data::::::::::::\n{classification_report(y_test, y_pred_oversampled, target_names=class_names)}")
print("\n-----\n")
print(f"Accuracy of undersampled data: {acc_undersampled:.3f}")
print(f"::::::::::::Classification Report of undersampled data::::::::::::\n{classification_report(y_test, y_pred_undersampled, target_names=class_names)}")
print("\n-----\n")

# Plot the confusion matrices for each model
fig, axes = plt.subplots(nrows=1, ncols=3, figsize=(15, 5))
fig.suptitle(f"{name} - Confusion Matrices", fontsize=16)

cm_orig = confusion_matrix(y_test, y_pred_orig)
sns.heatmap(cm_orig, annot=True, cmap="Blues", fmt="d", ax=axes[0])
axes[0].set_title("Original Data")

cm_oversampled = confusion_matrix(y_test, y_pred_oversampled)
sns.heatmap(cm_oversampled, annot=True, cmap="Blues", fmt="d", ax=axes[1])
axes[1].set_title("Oversampled Data")

cm_undersampled = confusion_matrix(y_test, y_pred_undersampled)
sns.heatmap(cm_undersampled, annot=True, cmap="Blues", fmt="d", ax=axes[2])
axes[2].set_title("Undersampled Data")

plt.show()

```

```
-----

                                Logistic Regression:
Accuracy of original data: 0.740
::::::::::::Classification Report of original data::::::::::::
              precision    recall  f1-score   support

         0           0.80       0.89       0.84        153
         1           0.41       0.26       0.32         47

    accuracy          0.74          0.74          0.74        200
   macro avg          0.60          0.57          0.58        200
weighted avg          0.71          0.74          0.72        200

-----

Accuracy of oversampled data: 0.720
::::::::::::Classification Report of oversampled data::::::::::::
              precision    recall  f1-score   support

         0           0.88       0.74       0.80        153
         1           0.44       0.66       0.53         47

    accuracy          0.72          0.72          0.72        200
   macro avg          0.66          0.70          0.66        200
weighted avg          0.77          0.72          0.74        200

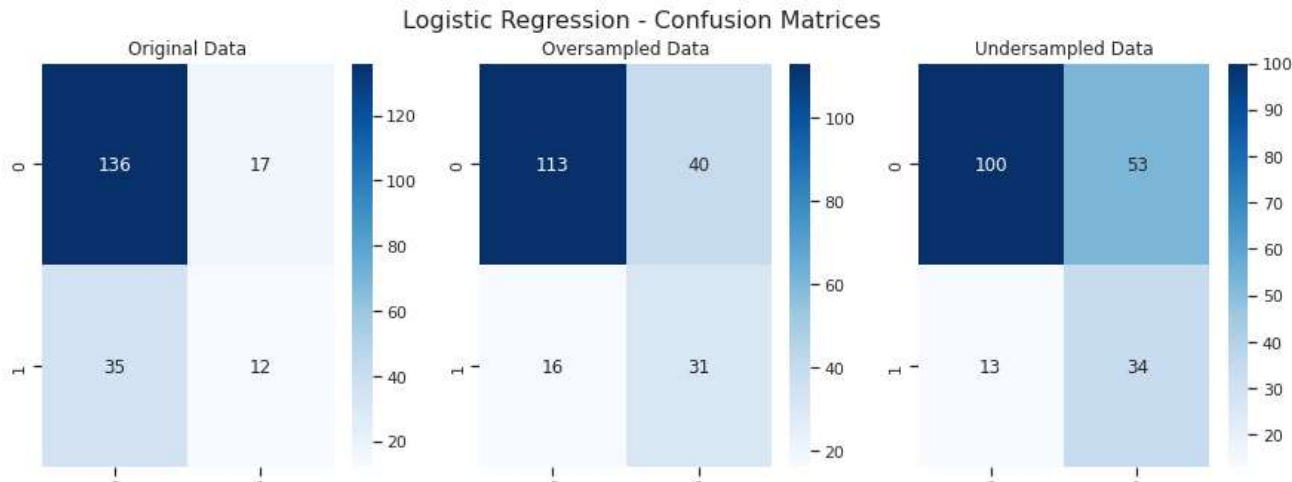
-----

Accuracy of undersampled data: 0.670
::::::::::::Classification Report of undersampled data::::::::::::
              precision    recall  f1-score   support

         0           0.88       0.65       0.75        153
         1           0.39       0.72       0.51         47

    accuracy          0.67          0.67          0.67        200
   macro avg          0.64          0.69          0.63        200
weighted avg          0.77          0.67          0.69        200

-----
```



Performing Cross - Validation by splitting in into 5 i.e cv =5

```

from sklearn.model_selection import cross_val_score

# create list of classifiers
classifiers = [LogisticRegression(), SVC(), DecisionTreeClassifier(), KNeighborsClassifier]

# create list of sampling methods
samplers = [('Original', None), ('SMOTE', SMOTE()), ('Random Under', RandomUnderSampler())]

# iterate through classifiers and samplers
for classifier in classifiers:
    for sampler_name, sampler in samplers:
        # if no sampler, use original data
        if sampler is None:
            x_resampled = X
            y_resampled = Y
        # else, resample data
        else:
            x_resampled, y_resampled = sampler.fit_resample(X, Y)

        # compute cross-validation scores for each combination of classifier and sampler
        scores = cross_val_score(classifier, x_resampled, y_resampled, cv=5)

        # print mean accuracy and standard deviation of cross-validation scores
        print("\n-----\n")
        print(f'{sampler_name} {type(classifier).__name__} cross-validation accuracy: {sco

```

Original SVC cross-validation accuracy: 0.749 +/- 0.016

SMOTE SVC cross-validation accuracy: 0.865 +/- 0.055

Random Under SVC cross-validation accuracy: 0.677 +/- 0.037

```

-----
SMOTE KNeighborsClassifier cross-validation accuracy: 0.669 +/- 0.031
-----

Random Under KNeighborsClassifier cross-validation accuracy: 0.583 +/- 0.020
-----

Original RandomForestClassifier cross-validation accuracy: 0.763 +/- 0.010
-----

SMOTE RandomForestClassifier cross-validation accuracy: 0.879 +/- 0.071
-----

Random Under RandomForestClassifier cross-validation accuracy: 0.795 +/- 0.030
-----

Original ExtraTreesClassifier cross-validation accuracy: 0.758 +/- 0.023
-----

SMOTE ExtraTreesClassifier cross-validation accuracy: 0.917 +/- 0.037
-----

Random Under ExtraTreesClassifier cross-validation accuracy: 0.740 +/- 0.035

```

The cross-validation scores for the models are presented above. Based on the disparity between the accuracy score and the cross-validation score, it can be inferred that the ExtraTrees Classifier is the most suitable model since it exhibits a significantly lower difference in comparison to the other models.

```

from sklearn.ensemble import ExtraTreesClassifier
from sklearn.pipeline import make_pipeline
from sklearn.impute import SimpleImputer
from sklearn.model_selection import GridSearchCV

# Create pipeline with an imputer and a classifier
pipe = make_pipeline(SimpleImputer(), ExtraTreesClassifier())

# Set the hyperparameters to tune
param_grid = {
    'extratreesclassifier__n_estimators': [50, 100, 200, 300],
    'extratreesclassifier__max_depth': [5, 10, 15, 20],
    'extratreesclassifier__min_samples_split': [2, 5, 10],
    'extratreesclassifier__min_samples_leaf': [1, 2, 4],
}

# Create the GridSearchCV object

```